



Structural Polyglot Orchestration, Native O-core, and World Execution

A technical account of the language, compiler, runtime,
and World architecture

Implementation snapshot

github.com/lostadi/Ostadix-lang

Source snapshot 36787b16476bc0c8c4ddf665c7228b314d04e716

Package coordinate 0.4.0 Unreleased

Lee Daghlar Ostadi

ORCID 0009-0001-6380-9558

25 August 2026

Abstract

I designed OSTADIX-LANG around a direct premise: the evaluator selected for an expression belongs in the expression's syntax. A hosted source file expresses that choice through balanced, language-indexed forms such as `python^(...)_python`. Nested evaluators are structural nodes, and their results cross language boundaries through OVALUE, a typed interchange algebra. The authoritative Rust runtime parses these forms into a small structural AST, lowers every normal execution through OIR, freezes backend metadata in a validated execution plan, projects the plan into a state-complete HGRAPH, and coordinates backend actors through canonical CBOR. Persistent language environments, quotation and meta-evaluation, deferred requests, coordination groups, project lifting, and hosted live-system supervision make evaluator composition a complete execution model.

I developed O-CORE as the native systems lineage of this architecture. It is a statically typed freestanding language whose AST-typed HIR-SSA MIR-x86-64 assembly/object pipeline supplies code used by the Ostadix kernel. Its compiler, runtime modules, capability system, and World-admission machinery carry the architecture from evaluator orchestration into native systems construction. Together, the hosted and native lineages span polyglot expression composition, state-complete execution graphs, freestanding compilation, and authority-bearing World execution.

In this paper I present the complete language system through one exact implementation snapshot. I explain the hosted grammar and semantics; the OVALUE interchange algebra; backend actors and authority; OIR, execution plans, and HGRAPH scheduling; project lifting and route execution; the O-CORE compiler; and the Live-World and KernelWorld layers. I then trace the current execution constitution through Graph V2, Evidence V6, Admission V6, Why V2, the unified lowercase `o` front door, reciprocal node pairing, Project Mesh, Information, Registry V1, and the canonical World formats. The central result is a coherent architecture in which evaluator identity is syntax, cross-language state is typed, effects become explicit graph resources, and execution authority is admitted through exact verified records.

Keywords: structural polyglot programming, evaluator composition, typed interchange values, hyper-graph execution, capability systems, freestanding systems language, multikernel research.

Core contributions

One language system, eight implemented contributions

1. **Evaluator-indexed syntax.** Every typed expression names its evaluator in the grammar, so heterogeneous source remains structurally visible in one recursive program tree.
2. **Typed inter-language state.** OVALUE gives every evaluator a common algebra for values, requests, groups, scopes, capabilities, and fidelity.
3. **State-complete graph execution.** OIR and HGRAPH represent values, completion, actor mutation, and world effects as explicit dependencies.
4. **Whole-project composition.** Deterministic project bundles preserve files, routes, guards, artifacts, and alternative policies in a reversible language artifact.
5. **Native World construction.** O-CORE, the kernel runtime, and KernelWorld carry the same explicit-structure philosophy into freestanding compilation and authority-bearing admission.
6. **Execution constituted by evidence.** Graph V2 binds the solved typed program; Evidence V6 binds its executable environment; Admission V6 rederives and materializes the authority consumed by execution.
7. **Capacity-first project placement.** Project Mesh moves a source-closed project branch with its prerequisites, ranks paired nodes by usable capacity, and records generation-fenced attempts.
8. **Information and World records.** The Information Kernel, eight bounded bridge projections, provenance admission, Registry V1, and the OWIDENT, OWPROTO, OWVALUE, and OWRECEIPT codecs give execution state canonical identities that can cross implementation boundaries.

Audience and reading paths

I wrote this paper for programming-language researchers, compiler and runtime engineers, systems and operating-system researchers, and technical readers evaluating Ostadix-lang as an execution substrate. It assumes familiarity with abstract syntax trees, intermediate representations, runtime processes, SSA, capability systems, and virtual-machine execution. Every Ostadix-specific concept is defined where it first enters the architecture.

The paper develops one continuous argument across three scales. The hosted language makes evaluator identity part of syntax and carries values between evaluators through a typed algebra. The compiler turns that structure into an effect-aware execution graph. O-CORE and the World layers carry the same explicit representation discipline into freestanding and authority-bearing execution.

Every reader can begin with Section 2, which presents the whole system and names its major layers. The paths below then follow each reader's primary technical question.

Reader	Recommended path	Central question
Programming-language researcher	Sections 4–6, Section 8, and Section 16.	How does evaluator-indexed syntax become a typed, compositional semantics?

Reader	Recommended path	Central question
Compiler or runtime engineer	Sections 3, 6–10, followed by Section 14.	How does source become a validated, scheduled, executable graph?
Distributed-runtime or agent-tool engineer	Sections 3, 9, and 14, followed by Section H.	How do local intent, reciprocal node trust, placement, Information, and World records preserve one execution identity?
Systems or OS researcher	Sections 11–13, followed by Sections 15 and 16.	How does the architecture reach freestanding code, capabilities, and World admission?
Technical adopter or evaluator	Sections 4, 9, 10, and 14, followed by the compact reference and reproduction appendices.	What can the implementation express, build, inspect, and execute today?

How to read the notation, figures, and evidence

Monospaced text denotes source forms, commands, evaluator identities, resource names, and emitted compiler labels. `OVALUE`, `OIR`, `HGRAPH`, `HostWorld`, and `ActorState` name concrete implementation types or compiler concepts. Architecture figures generally flow from source on the left toward execution or publication on the right. The complete graph in Section 9 has its own visual grammar and a seven-pass reading method immediately before the compiler-emitted figure.

Source citations name the implementation file and line interval that realizes the surrounding mechanism. Validation tables identify the command, observed result, and validated surface for the implementation release. This lets a reader move from thesis, to mechanism, to defining code, to executable evidence without changing terminology.

Contents

Audience and reading paths	ii
1 Implementation basis and method	1
1.1 Source coordinate	1
1.2 Contribution map	1
1.3 Architectural path through the implementation	1
2 The language system in one view	2
2.1 Hosted orchestration	2
2.2 Project execution	2
2.3 Native systems execution	2
2.4 World layers	2

3	The current execution constitution	3
3.1	Independent <code>ostadix-api</code> ownership	3
3.2	The unified lowercase <code>o</code> front door	3
3.3	Graph V2, Evidence V6, Admission V6, and Why V2	4
3.4	The ten-tool Ostadix MCP	5
3.5	Reciprocal LAN pairing and pinned TLS	6
3.6	Capacity-first Project Mesh	7
3.7	Information Kernel, Bridge, and Provenance	8
3.8	Registry V1	9
3.9	World identities, values, protocols, and receipts	9
4	Hosted surface language	10
4.1	Implemented structural grammar	10
4.2	First example: evaluator-indexed expressions	12
4.3	Bindings and dependencies	12
4.4	Persistent and ephemeral evaluators	13
5	Hosted dynamic semantics	13
5.1	Evaluation order	13
5.2	Quotation and meta-evaluation	14
5.3	Demand policies and requests	14
5.4	Coordination groups	15
6	The OValue semantic boundary	16
6.1	Authorable tagged OValue schema	17
6.2	Fidelity	18
6.3	Representation-sensitive identity	18
6.4	Wire protocol	19
7	Backends, actors, and authority	19
7.1	Static registry	19
7.2	Actor process model	20
7.3	Hosted authority model	20
8	OIR, plans, effects, and HGraph execution	21
8.1	OIR is orchestration IR	21
8.2	Execution plan	21
8.3	State-complete HGraph	22
8.4	Effects and explicit resource state	22

9 Projects, linking, and route policies	23
9.1 Project lifting and literal linking	23
9.2 Compiler-emitted Project HGraph from an o-linked codebase	23
9.3 Bundle model	31
9.4 Routes	31
9.5 Unlinking	33
10 Toolchain and implementation editions	34
10.1 Public binaries	34
10.2 What olang targets mean	34
10.3 Reference implementations	35
10.4 REPL and notebook state	36
10.5 O-Git	36
11 O-core: the native language	37
11.1 Language surface	37
11.2 Static semantics	38
11.3 Compiler pipeline	39
11.4 Current compiler profile	39
12 Kernel, capabilities, and native execution	40
12.1 Freestanding kernel substrate	40
12.2 Native capability model	40
13 Live-World and KernelWorld	42
13.1 Three different layers	42
13.2 Live-World command and state model	43
13.3 Authorable Live-World schemas	43
13.4 KernelWorld authority chain	46
13.5 Native execution ladder	48
13.6 Mode 20 and Mode 21	49
13.7 Modes 22 through 34	50
14 Reproducibility and current-master evidence	51
14.1 Evidence classes	52
14.2 Current snapshot checks	52
14.3 Exact reproduction commands	54
14.4 Coverage accounting	55

15 Implemented capabilities	55
16 Design rationale	56
16.1 Why evaluator identity is syntax	56
16.2 Why graph state is explicit	56
16.3 Why analysis and authority are separate objects	56
16.4 Why the engine owns the types	56
16.5 Why admission is typed across layers	57
16.6 Why placement begins with capacity	57
17 Conclusion	57
A Compact hosted-language reference	57
A.1 Exact builtin signatures	58
B Public command reference	59
C Example atlas	61
D Source map	62
E Complete ostadix-api module inventory	64
F Architectural position among neighboring systems	65
G Reproduction commands	65
G.1 Pin the source and qualify the Rust workspace	66
G.2 Run the language and unified front door	66
G.3 Qualify the independent engine and MCP	66
G.4 Generate and read the o-linked Project HGraph	67
G.5 Run Project HGraph, Live-World, and World gates	67
G.6 Run compiler targets	67
G.7 Run the Python and C17 editions	68
G.8 Inspect Information, Registry, and node surfaces	68
H Current host, integration, and proof boundaries	68

1 Implementation basis and method

1.1 Source coordinate

This paper describes the synchronized Ostadix-lang package coordinate **0.4.0 Unreleased** at source snapshot **36787b16**. The title page records the complete commit identity. I tie every technical claim to implementation code, compiler-generated artifacts, executable tests, or declared evidence gates from this snapshot.

The workspace contains two synchronized Rust packages. The root **o-lang** package owns compatibility reexports and command-line programs. The independently packageable **ostadix-api** package owns the parser, IR, evaluator, graph, evidence, scheduler, project, networking, Information, Registry, World, Live-World, KernelWorld, and O-core runtime modules. The MCP server is a separately locked Rust package excluded from the main workspace so its protocol dependencies remain outside the runtime dependency graph. [Cargo.toml, lines 1–18] [crates/ostadix-api/Cargo.toml, lines 1–16] [CHANGELOG.md, lines 8–50]

1.2 Contribution map

Contribution	Implementation mechanism	What it enables
Structural polyglot syntax	Exact evaluator tags, recursive parser nodes, structural splices, and persistent actor identities.	One program tree can contain native Python, SQL, Nix, shell, markup, and other language regions.
Typed exchange	OVALUE, canonical CBOR, renderer contracts, and composed fidelity states.	Values retain explicit meaning and conversion history while crossing evaluators.
Graph semantics	OIR, validated plans, hyperedges, completion tokens, and versioned resource states.	Execution order, effects, mutation, and safe parallelism become inspectable program structure.
Project composition	Lossless bundles, materialization, route policies, guards, and artifact collection.	Whole codebases become reversible, executable Ostadix artifacts.
Native World lineage	O-CORE HIR/MIR/code generation, kernel capabilities, exact records, and generation-bound admission.	The language architecture reaches from hosted composition to machine-level execution and lifecycle authority.

1.3 Architectural path through the implementation

The paper follows the implementation from source syntax to execution:

1. parse hosted syntax into **ONode**;
2. lower it to **OIR**, a validated execution plan, and **HGRAPH**;
3. trace effects, actor identity, scheduling, process reuse, wire values, and authority;
4. trace project bundling and route execution separately;
5. trace **O-CORE** from lexer through object generation;

6. trace hosted Live-World, native kernel gates, KernelWorld admission, and SVM execution;
7. rebuild and run the complete applicable gate matrix on the exact snapshot.

2 The language system in one view

OSTADIX-LANG is best understood as a family with two compiler lineages and several execution substrates.

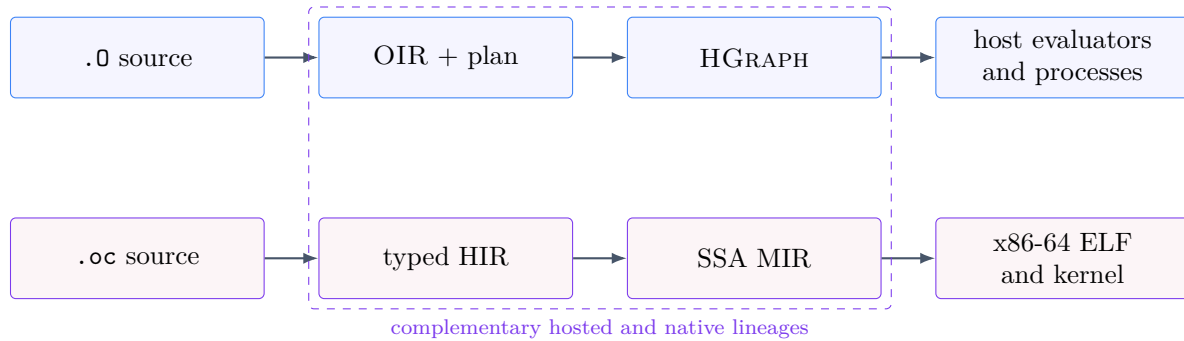


Figure 1: Compiler bifurcation. Hosted `.O` orchestrates evaluators; `.oc` compiles machine computation.

[docs/OCORE.md, lines 3–30]

2.1 Hosted orchestration

Hosted O is a compact structural meta-language built from evaluator-indexed blocks, structural nesting and splicing, bindings, orchestration calls, persistent evaluator identities, quotation, explicit scopes, and demand policies. Computation remains native to its selected evaluator while O supplies the shared program structure.

This design makes Python, SQL, Nix, Markdown, shell, and other runtimes direct participants in one structurally visible program tree, with each evaluator retaining its own language semantics.

2.2 Project execution

Project execution lifts a directory into a lossless **ProjectBundle**. A dedicated subprocess coordinator executes its routes, dependencies, guards, artifacts, and alternative policies while the bundle preserves the codebase as one reversible language artifact.

2.3 Native systems execution

O-CORE is a statically typed systems language with modules, functions, structures, enums, explicit unsafe operations, inline assembly, and freestanding ABI constraints. Its output can be linked into the Ostadix kernel. The native lineage gives Ostadix-lang a machine-level language for implementing the runtime substrate that hosted O can construct, package, and coordinate.

2.4 World layers

Hosted Live-World provides executable local-process lifecycle semantics. Native kernel modes realize capability-governed services and virtualized execution. KernelWorld binds exact packages, manifests,

policy, architecture, authority, health, and either package-payload image bytes or the declared SHA-256 of a user-supplied image into a verified admission chain. These layers form successive realizations of the World model.

3 The current execution constitution

The current master makes execution a composed constitutional path. Source becomes a solved graph. The analyzer binds that graph to runtime facts. The admission compiler reproduces those facts and materializes the inputs that the coordinator consumes. Local commands, agent tools, and Project Mesh all enter that path through explicit records. Information and World formats preserve the identities needed to inspect, move, and verify the resulting computation.

3.1 Independent `ostadix-api` ownership

I separated implementation ownership from command-line packaging. The publishable `ostadix-api` crate owns the runtime engine and its nominal types. The root `o-lang` package depends on the exact synchronized engine version and reexports the engine modules used by established `o_lang::` callers. A value parsed through the root library and a value handled through `ostadix_api` therefore have one defining type and one runtime implementation.

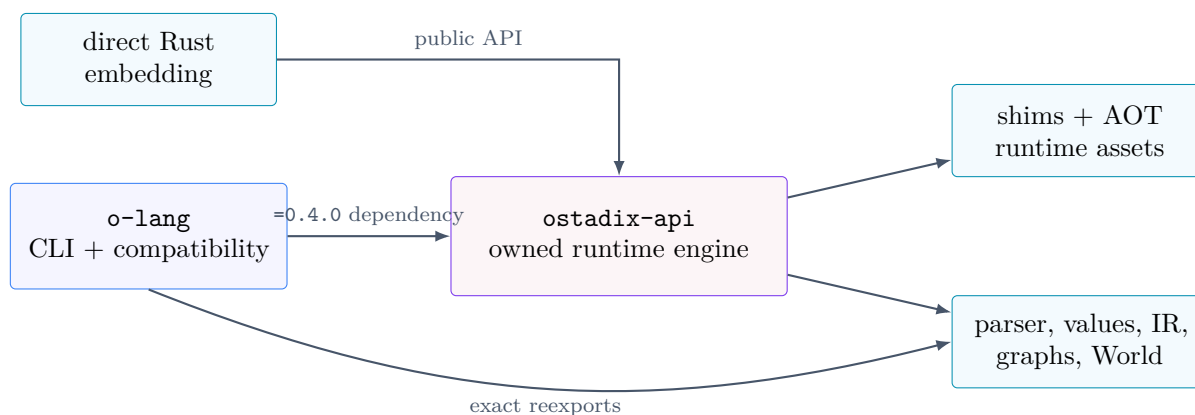


Figure 2: One engine owns runtime behavior. The root package supplies commands and preserves established imports by reexporting the engine’s exact types.

The ownership boundary is enforced by the workspace and package manifests, the engine module root, and the root’s one-way reexport list. `[Cargo.toml, lines 14–18]` `[Cargo.toml, lines 82–86]` `[crates/ostadix-api/Cargo.toml, lines 1–16]` `[crates/ostadix-api/src/lib.rs, lines 1–59]` `[src/lib.rs, lines 1–13]`

3.2 The unified lowercase `o` front door

The lowercase `o` command is the language’s intent front door. Its compiled path implements `run`, `plan`, `explain`, and `inspect`; the repository dispatcher routes those verbs into `o-cli` and retains the remaining operational command families. The front door classifies an input as hosted O source, a directory, a lifted project, or an accepted standalone form, then selects the ordinary evaluator or project engine and renders one stable intent record.

```

o run program.0
o plan program.0

```

```

o explain last-run
o inspect last-run

o run ./project --route main
o plan ./project --route main

```

Each completed run yields a durable `RunRecordV1`. The record binds a schema, run identity, wall-clock time, resolved intent, engine selection, source identity, execution result, and optional trace attachment. The store uses canonical JSON bytes, validates every record before acceptance, writes through a temporary file, renames atomically, and maintains the selected history root. Record identity is content-bound, so inspection names the artifact that was actually persisted.

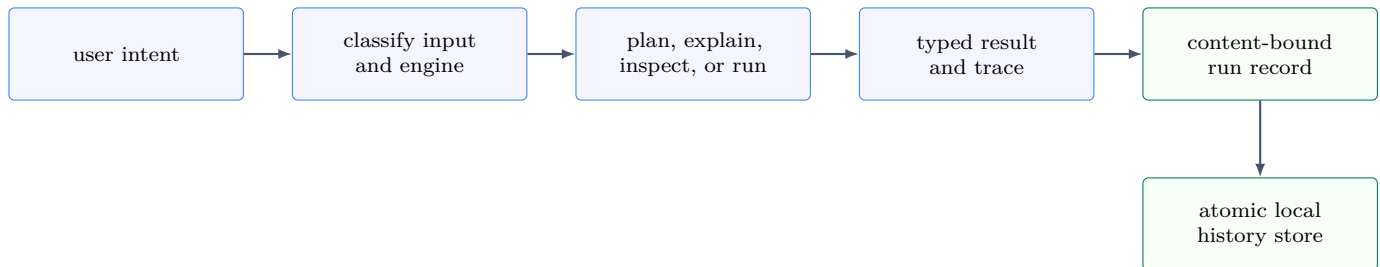


Figure 3: The lowercase front door converts command intent into a resolved execution path and a durable record that can be inspected later.

[scripts/o-cli.sh, lines 18–26] [src/bin/o-cli.rs, lines 42–68] [src/bin/o-cli.rs, lines 403–410] [src/bin/o-cli.rs, lines 472–517]
 [src/bin/o-cli.rs, lines 931–990] [src/bin/o-cli.rs, lines 992–1066] [src/bin/o-cli.rs, lines 1641–1676] [crates/ostadix-api/
 src/intent/record.rs, lines 25–29] [crates/ostadix-api/src/intent/record.rs, lines 53–100] [crates/ostadix-api/src/intent/
 record.rs, lines 894–955] [crates/ostadix-api/src/intent/store.rs, lines 1–6] [crates/ostadix-api/src/intent/store.rs, lines
 27–35] [crates/ostadix-api/src/intent/store.rs, lines 84–144]

3.3 Graph V2, Evidence V6, Admission V6, and Why V2

Graph V2 is the executable identity of a solved hosted program. Its canonical digest covers graph nodes, typed values, actor identities, the complete typed fidelity assessment, executable operations, effects, sequence dependencies, roots, and bindings. A semantic change in any of those fields produces a different graph coordinate.

Evidence V6 binds that graph to the complete lowering and execution context. The bundle carries the OIR and plan identities, Graph V2 identity, analyzer coordinate, backend Catalog V5 projection, backend artifact bindings, runtime binding, environment binding, ambient World binding, and one typed assessment for every executable operation. The analyzer first validates the caller graph against a freshly solved Graph V2 before emitting the bundle.

Admission V6 is the executable authority compiler. It checks the schema and analyzer coordinate, re-solves and re-analyzes the graph, matches the complete operation inventory, verifies every hard assessment, materializes admission evidence nodes, validates the admitted graph, and constructs its ready schedule. The coordinator and evaluator accept the resulting `AdmittedExecution` object.

Why V2 is the reader-facing explanation of that authority. For a selected plan node it reports the admitted operation, its inputs and outputs, effect and actor rails, dependency witnesses, typed fidelity decision, and schedule position. It is derived from Admission V6 as an inspection operation.

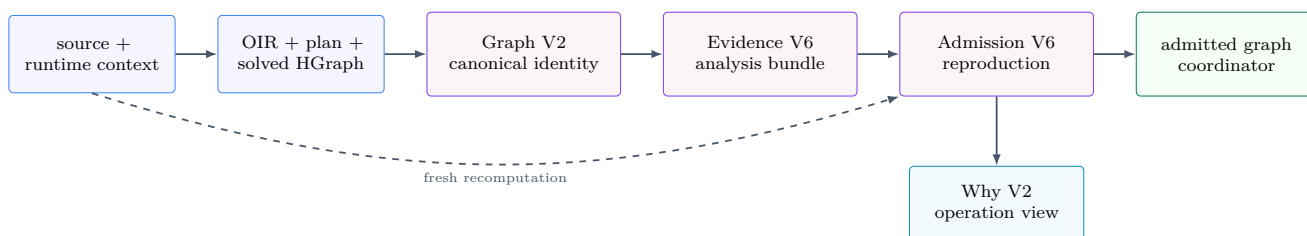


Figure 4: Current hosted execution is a proof-producing and proof-consuming pipeline. Admission reproduces the analyzer result before execution begins.

Coordinate	Primary object	Mechanism
Graph V2	Solved executable HGRAPH	Canonical identity over executable shape, typed values, effects, actor state, fidelity, roots, and bindings.
Evidence V6	<code>EvidenceBundleV6</code>	Analyzer result bound to graph, lowering, catalog, artifacts, runtime, environment, and World.
Admission V6	<code>AdmittedExecution</code>	Fresh reproduction, complete per-operation validation, evidence materialization, and ready scheduling.
Why V2	<code>ScheduleWhyV2</code>	Stable operation neighborhood projected from the admitted schedule for human and tool inspection.

[crates/ostadix-api/src/evidence/fact.rs, lines 10–15] [crates/ostadix-api/src/evidence/fact.rs, lines 460–493] [crates/ostadix-api/src/evidence/analyze.rs, lines 33–36] [crates/ostadix-api/src/evidence/analyze.rs, lines 239–272] [crates/ostadix-api/src/evidence/analyze.rs, lines 624–676] [crates/ostadix-api/src/evidence/analyze.rs, lines 702–769] [crates/ostadix-api/src/evidence/analyze.rs, lines 792–809] [crates/ostadix-api/src/evidence/analyze.rs, lines 818–953] [crates/ostadix-api/src/evidence/admit.rs, lines 105–114] [crates/ostadix-api/src/evidence/admit.rs, lines 195–231] [crates/ostadix-api/src/evidence/admit.rs, lines 274–330] [crates/ostadix-api/src/evidence/admit.rs, lines 1882–1989] [crates/ostadix-api/src/evidence/admit.rs, lines 2201–2335] [src/bin/olangc.rs, lines 1332–1372]

3.4 The ten-tool Ostadix MCP

The Ostadix MCP is part of this master snapshot. It is a standalone Rust/`rmcp` stdio server with its own lockfile, built by default through `setup.sh` and registered by the repository’s `.mcp.json`. Its ten tools expose environment, runtime, Information, compilation, execution, and diagnostic operations through resolved repository and command-line contracts.

Tool	Owned operation
<code>o_env</code>	Report the resolved repository, backend, binary, and runtime environment used by subsequent tools.
<code>o_runtimes</code>	Enumerate every canonical evaluator backend and report its executable requirements and current local availability.

Tool	Owned operation
<code>o_information_inspect</code>	Read the current Information root through the bounded existing-root inspection path.
<code>o_smoke</code>	Execute the canonical hosted smoke program and report its result.
<code>o_run</code>	Run a hosted input through the direct evaluator path.
<code>o_analyze_intent</code>	Resolve an intent, compute current analysis, and return a bounded execution handle.
<code>o_execute_intent</code>	Consume one analysis handle and ask the root O process to reproduce and admit the execution.
<code>o_olangc</code>	Invoke the compiler's IR, DOT, script, binary, or WASM surface with checked arguments.
<code>o_doctor</code>	Diagnose repository, backends, binaries, and runtime availability.
<code>o_search_run</code>	Run a named a18re search tool from an explicitly selected work directory, <code>A18_WORK</code> , or the local a18re default, using the resolved O executable and backend directory.

Analyze then execute forms a one-use authorization protocol. The server binds the handle to program bytes, working directory, repository root, backend directory, source identity, and resolved intent. It gives the entry a bounded lifetime and removes it before execution validation. The root O process receives the bound request, recomputes current Graph V2 and Evidence V6, and constructs fresh Admission V6 authority. A successful response therefore connects the agent-visible intent to the same current execution constitution used by the language runtime.

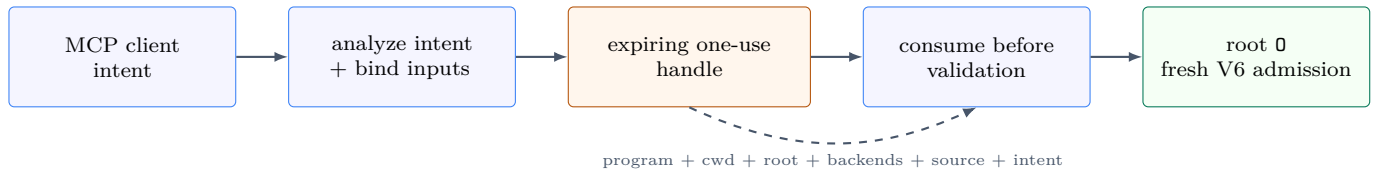


Figure 5: The MCP analyze/execute protocol carries a bounded intent binding into a fresh root-runtime admission.

[mcp/ostadix_lang_mcp_server/Cargo.toml, lines 1–12] [mcp/ostadix_lang_mcp_server/Cargo.toml, lines 26–32] [setup.sh, lines 886–910] [setup.sh, lines 1582–1586] [.mcp.json, lines 1–7] [mcp/ostadix_lang_mcp_server/src/main.rs, lines 43–68] [mcp/ostadix_lang_mcp_server/src/main.rs, lines 77–187] [mcp/ostadix_lang_mcp_server/src/main.rs, lines 1663–1766] [mcp/ostadix_lang_mcp_server/src/main.rs, lines 1768–1864] [mcp/ostadix_lang_mcp_server/src/main.rs, lines 1858–2015] [mcp/ostadix_lang_mcp_server/src/main.rs, lines 2018–2207] [mcp/ostadix_lang_mcp_server/src/main.rs, lines 2209–2222] [src/main.rs, lines 81–128] [src/main.rs, lines 161–225] [crates/ostadix-api/src/eval.rs, lines 2455–2506]

3.5 Reciprocal LAN pairing and pinned TLS

Node trust begins with a reciprocal, one-use, ten-digit pairing code. Each node creates an ephemeral SPAKE2 role and validates the peer transcript. Directional HKDF and HMAC keys bind the code, both node identities, both endpoints, both nonces, and the full request/response transcript. Each destination then issues the client certificate it will accept for the peer. Private keys remain local; the stored peer record carries the destination CA, destination server name, client certificate, and generation needed for the next TLS session.

The data plane uses TLS 1.3 with the Ostadix ALPN token. Outbound connection validation loads the paired destination CA and checks the stored server name. The node listener validates client certificates against its pairing CA before the application protocol receives a request. Discovery supplies addresses; the paired record supplies identity and cryptographic continuity.

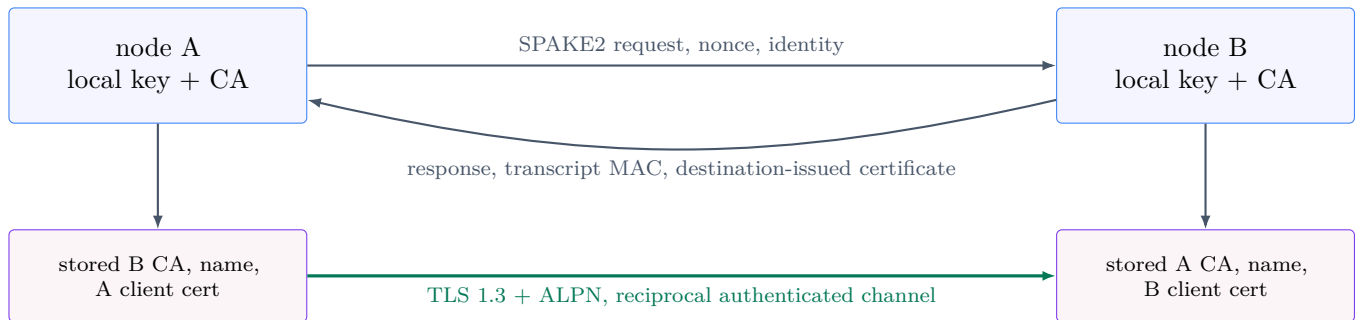


Figure 6: Pairing turns a short shared code into reciprocal destination pins and client credentials. Those persisted facts establish later node channels.

[crates/ostadix-api/src/hosted_remote/pairing.rs, lines 1–7] [crates/ostadix-api/src/hosted_remote/pairing.rs, lines 24–76]
 [crates/ostadix-api/src/hosted_remote/pairing.rs, lines 79–193] [crates/ostadix-api/src/hosted_remote/pairing.rs, lines 195–387]
 [crates/ostadix-api/src/hosted_remote/pairing.rs, lines 389–586] [crates/ostadix-api/src/hosted_remote/pairing.rs, lines 684–815]
 [crates/ostadix-api/src/hosted_remote/lan.rs, lines 693–870] [src/bin/o-node.rs, lines 1070–1175] [src/bin/oct1.rs, lines 874–970]
 [crates/ostadix-api/src/hosted_remote/tls.rs, lines 61–156] [crates/ostadix-api/src/hosted_remote/tls.rs, lines 204–303]

3.6 Capacity-first Project Mesh

Project Mesh distributes a project at the route-branch scale. The planner selects the requested route branch and closes it over prerequisite routes, inputs, declared artifacts, and bundle provenance. Candidate nodes come from paired authenticated discovery and carry current profile, compatibility, and capacity facts. Ranking places usable execution capacity first, then applies the route and policy constraints that preserve semantic compatibility.

The selected node receives a source-closed project payload and a bounded execution request. Generation fences bind discovery, reservation, attempt, and result state. Retry contracts record why an attempt can continue, while concurrent selection policies record all relevant attempts and their terminal settlement. Mesh traces therefore connect placement decisions to the exact project branch, node generation, and returned artifacts.

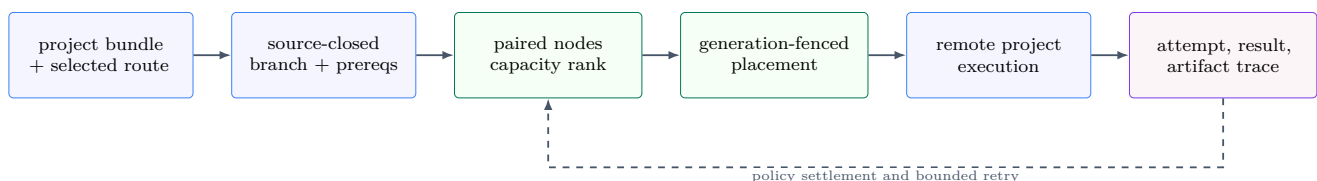


Figure 7: Project Mesh moves a complete route branch, chooses capacity, binds node generations, and returns a causally connected attempt trace.

[crates/ostadix-api/src/hosted_remote/project_mesh.rs, lines 1–6] [crates/ostadix-api/src/hosted_remote/project_mesh.rs, lines 45–112]
 [crates/ostadix-api/src/hosted_remote/project_mesh.rs, lines 278–375] [crates/ostadix-api/src/hosted_remote/project_mesh.rs, lines 804–895]
 [crates/ostadix-api/src/hosted_remote/project_mesh.rs, lines 1024–1117] [crates/ostadix-api/src/hosted_remote/project_mesh.rs, lines 1810–2018]
 [crates/ostadix-api/src/hosted_remote/project_mesh.rs, lines 2092–2243]

[crates/ostadix-api/src/hosted_remote/project_mesh.rs, lines 2260–2335] [crates/ostadix-api/src/hosted_remote/mesh.rs, lines 1510–1719]

3.7 Information Kernel, Bridge, and Provenance

The Information Kernel turns observations into content-addressed information state. An entity descriptor identifies what a fact is about. An Information Atom binds that entity, a typed payload tier, assertions, and provenance. Snapshots assemble atom identities into one immutable view. Revisions advance named roots through compare-and-swap, so a reader can identify the exact view it observed and a writer can prove the predecessor it replaced.

Payload tiers keep size and meaning explicit. T0 carries bounded public scalars. T1 identifies content held by the managed blob store. T2 identifies external content through canonical schema, media type, digest, and logical length. Canonical CBOR and domain-separated SHA-256 give every atom, blob, snapshot, and revision a stable identity.

Information Bridge V1 defines exactly eight bounded projections:

1. parsed document identity and structural counts;
2. allowlisted public scalar value identity;
3. solved HGraph metadata;
4. exact Evidence V6 metadata;
5. verified Registry V1 profile metadata;
6. verified World receipt identity and semantic digest;
7. logical project graph identity; and
8. Hosted V2 journal-entry identity.

Each projection has a private wire shape, strict schema check, bounded canonical-CBOR decoder, and canonical re-encode check. The bridge chooses the metadata that belongs in Information and hashes identifiers whose native locator vocabulary should remain outside the Information record.

Information Provenance V2 adds a content-addressed sidecar over frozen atom identities. Typed witnesses describe acquisition, producer authentication, procedure, signer authorization, receipt currentness, plan-node attribution, execution and effect fidelity, and morphism fidelity. Admission recomputes the sidecar image, verifies the bound World receipt and execution evidence, then derives a recovery assessment with intrinsic loss, discharged obligations, and unresolved obligations. Exact recovery establishes an origin statement that can be presented to Information readers.

[crates/ostadix-api/src/information/mod.rs, lines 1–7] [crates/ostadix-api/src/information/mod.rs, lines 22–60] [crates/ostadix-api/src/information/model.rs, lines 20–38] [crates/ostadix-api/src/information/model.rs, lines 143–245] [crates/ostadix-api/src/information/model.rs, lines 275–298] [crates/ostadix-api/src/information/model.rs, lines 354–410] [crates/ostadix-api/src/information/model.rs, lines 448–500] [crates/ostadix-api/src/information/root.rs, lines 5–123] [crates/ostadix-api/src/information/provenance.rs, lines 1–8] [crates/ostadix-api/src/information/provenance.rs, lines 119–213] [crates/ostadix-api/src/information/provenance.rs, lines 424–475] [crates/ostadix-api/src/information/provenance.rs, lines 721–810] [crates/ostadix-api/src/information_bridge/mod.rs, lines 1–6] [crates/ostadix-api/src/information_bridge/mod.rs, lines 25–54] [crates/ostadix-api/src/information_bridge/mod.rs, lines 74–164] [crates/ostadix-api/src/information_bridge/mod.rs, lines 283–317] [crates/ostadix-api/src/information_bridge/mod.rs, lines 527–601] [crates/ostadix-api/src/information_bridge/mod.rs, lines 871–1037] [crates/ostadix-api/src/information_provenance/mod.rs, lines 1–7] [crates/ostadix-api/src/information_provenance/mod.rs, lines 28–38] [crates/ostadix-api/src/information_provenance/mod.rs, lines 74–194] [crates/ostadix-api/src/information_provenance/mod.rs, lines 330–505]

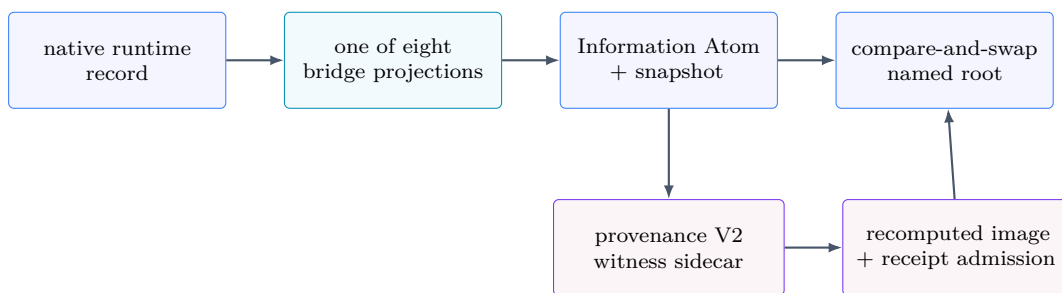


Figure 8: Native facts enter the Information Kernel through explicit bounded projections; provenance sidecars add typed recovery evidence to frozen atom identity.

3.8 Registry V1

Registry V1 is an append-only Ed25519 namespace and node-profile chain. A namespace root establishes the trust anchor and validity interval. A root or delegate can authorize a strict descendant namespace. Profile publications bind a namespace, node identity, generation-bearing node profile, issue time, and expiry. Every event carries its sequence, predecessor digest, namespace, signer key, and signed body.

Verification walks the chain from configured anchors, validates signatures and delegation scope, enforces time bounds and profile freshness, and maintains the highest accepted generation for each node. Sequence, predecessor, rollback, and equivocation checks turn a local event file into a verifiable publication history. The resulting `VerifiedRegistryProfileV1` is the profile form consumed by placement and Information projection.

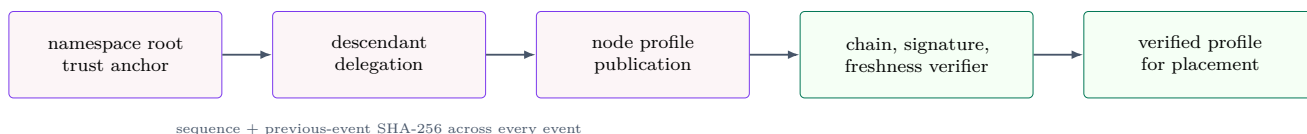


Figure 9: Registry V1 turns signed namespace events into a freshness-checked node profile suitable for placement decisions.

[crates/ostadix-api/src/registry/mod.rs, lines 1–5] [crates/ostadix-api/src/registry/mod.rs, lines 18–67] [crates/ostadix-api/src/registry/model.rs, lines 9–22] [crates/ostadix-api/src/registry/model.rs, lines 24–74] [crates/ostadix-api/src/registry/model.rs, lines 76–152] [crates/ostadix-api/src/registry/model.rs, lines 154–245] [crates/ostadix-api/src/registry/model.rs, lines 299–340] [crates/ostadix-api/src/registry/crypto.rs, lines 14–60] [crates/ostadix-api/src/registry/crypto.rs, lines 70–225] [crates/ostadix-api/src/registry/verify.rs, lines 55–226] [crates/ostadix-api/src/registry/verify.rs, lines 227–348] [crates/ostadix-api/src/registry/verify.rs, lines 393–442]

3.9 World identities, values, protocols, and receipts

The World layer uses four separate canonical formats. Separation keeps each decoder’s vocabulary, size ceiling, and authority meaning explicit.

Format	Primary unit	Mechanism
OWIDENT V1	World identity atom	Deterministic identity-only records with strict nonzero generation, term, version, and index rules plus hierarchical stale and logical comparison.
OWPROTO V1	Bounded protocol record	Big-endian framing, four fixed record kinds, exact-length and reserved-field validation, and deterministic version and record-limit negotiation.
OWVALUE V1	Portable value	Self-framed allowlisted value tree with bounded bytes, depth, and nodes; ordered records and scalar-key maps; root extension envelope with a portable payload.
OWRECEIPT V1	Execution receipt	Canonical execution identity, World generations, content digests, descriptive rights, terminal and commit fields, evidence gate, and domain-separated Ed25519 signature envelope.

The Rust and O-core implementations share the wire definitions and canonical corpora. Receipt verification returns a typed verified receipt. Semantic projection removes signer-dependent envelope variation so hosted execution and native Mode 32 can compare the same execution meaning.

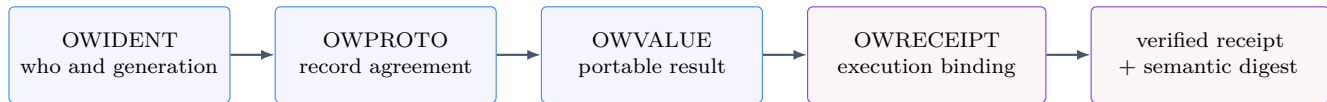


Figure 10: The World formats compose identity, protocol, portable value, and signed execution evidence while retaining separate canonical envelopes.

[crates/ostadix-api/src/world/identity_wire.rs, lines 18–39] [crates/ostadix-api/src/world/identity_wire.rs, lines 62–77]
 [crates/ostadix-api/src/world/identity_wire.rs, lines 98–249] [crates/ostadix-api/src/world/protocol.rs, lines 10–12] [crates/ostadix-api/src/world/protocol.rs, lines 148–212] [crates/ostadix-api/src/world/codec.rs, lines 17–55] [crates/ostadix-api/src/world/codec.rs, lines 67–115] [crates/ostadix-api/src/world/codec.rs, lines 118–233] [crates/ostadix-api/src/world/value_codec.rs, lines 1–6] [crates/ostadix-api/src/world/value_codec.rs, lines 22–51] [crates/ostadix-api/src/world/value_codec.rs, lines 88–143] [crates/ostadix-api/src/world/value_codec.rs, lines 221–340] [crates/ostadix-api/src/world/receipt.rs, lines 1–6] [crates/ostadix-api/src/world/receipt.rs, lines 20–28] [crates/ostadix-api/src/world/receipt.rs, lines 600–769] [crates/ostadix-api/src/world/receipt.rs, lines 806–839] [crates/ostadix-api/src/world/receipt_codec.rs, lines 27–40] [crates/ostadix-api/src/world/receipt_codec.rs, lines 56–131] [crates/ostadix-api/src/world/receipt_codec.rs, lines 152–227] [crates/ostadix-api/src/world/receipt_codec.rs, lines 230–326]

4 Hosted surface language

4.1 Implemented structural grammar

The implementation AST has five semantic forms: `RawText`, `VarRef`, `LetBinding`, `TypedExpr`, and `Call`. [crates/ostadix-api/src/parser.rs, lines 22–59] The following EBNF is an implementation-oriented summary; context rules below are part of the grammar.

```

cli_source    ::= [ shebang newline ] document
shebang      ::= "#" raw_line
document     ::= sequence
sequence     ::= { comment | raw | var | typed | binding | call }

typed        ::= raw_tag "(" body ")" raw_tag
raw_tag      ::= ident [ "[" uint "]" ] [ attributes ]
attributes   ::= "{" attribute { "," attribute } "}"
attribute    ::= ident | ident "=" visible_value
visible_value ::= visible_attr_char { visible_attr_char }

var          ::= "$" ident
binding      ::= "let" ident "=" ( typed | call )
call         ::= ident "(" [ call_arg { "," call_arg } ] ")"
call_arg     ::= var | call
comment      ::= "#" { non_newline_byte } [ newline ]

ident        ::= ( letter | "_" ) { letter | digit | "_" }
uint         ::= digit { digit }
letter       ::= "A"..."Z" | "a"..."z"
digit        ::= "0"..."9"

```

`visible_attr_char` is visible ASCII excluding braces, comma, and equals. `raw` is one or more source bytes up to the next unescaped structural token recognized in the current context; `raw_line` and `non_newline_byte` end at LF or CR/LF. At document level and inside `0` or `quote`, the recognized tokens are comments, bindings, calls, variables, and registered typed openers. Inside every other evaluator body, only variables and registered typed openers are structural. A backslash escapes an otherwise structural opener, closer, or variable splice. The CLI entrypoints strip one leading shebang before invoking the parser.

4.1.1 Exact tag symmetry

The parser closes a typed expression only when its language, environment, and attribute text exactly reproduce the opener. This lexical symmetry makes the extent of every evaluator invocation locally decidable, even when foreign source contains parentheses of its own. A missing environment is represented internally by `u32::MAX` and selects a fresh ephemeral actor. An explicit `LANG[n]` selects a persistent evaluator actor. Language aliases canonicalize in the AST, and reconstruction emits the canonical form.

[crates/ostadix-api/src/parser.rs, lines 590–738] [crates/ostadix-api/src/parser.rs, lines 1030–1120]

4.1.2 Structural versus opaque contexts

`$name` and registered nested typed blocks are recognized throughout foreign bodies. The sequencing forms `let`, calls, and `#` comments are recognized at document level and inside `0` and `quote`. Everywhere else, the parser preserves backend source verbatim. Registry membership therefore controls which apparent tag forms become structural nodes, while ordinary backend syntax retains its native meaning.

[crates/ostadix-api/src/parser.rs, lines 6–20] [crates/ostadix-api/src/parser.rs, lines 215–410]

4.1.3 Intentional smallness

`O` delegates literal construction and ordinary computation to its evaluators. A binding right-hand side is a typed block or call, and a call argument is a variable load or nested call. Every resulting value therefore

has an explicit origin in an evaluator or orchestration builtin, which keeps the host grammar small while preserving the full expression systems of its member languages. [crates/ostadix-api/src/parser.rs, lines 412–561]

4.2 First example: evaluator-indexed expressions

```
html^(
  <p>The answer is python^(
    __oval_result__ = sum(x*x for x in range(10))
  )_python.</p>
)_html
```

The inner Python evaluator produces a typed result. Before the outer HTML backend runs, the runtime renders that result according to HTML’s splice rules. The splice is a first-class child node with its own evaluator identity, environment, result type, and data dependency. This is the central syntactic move: foreign source remains idiomatic while cross-language composition is visible to parsing, planning, and scheduling.

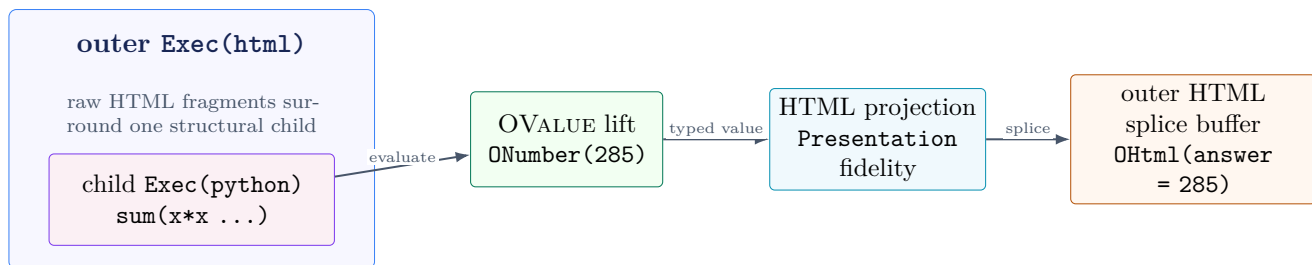


Figure 11: A nested evaluator is a typed child in the program tree. Its result crosses the evaluator boundary as OVALUE, then the receiving inline HTML projection assembles the outer OHtml value.

4.3 Bindings and dependencies

```
let answer = python^(
  __oval_result__ = 40 + 2
)_python

python^(
  __oval_result__ = $answer + 1
)_python
```

let lowers to Store; \$answer lowers to Load. Plan construction connects the load to the most recent visible store with an explicit data edge. Each shim also receives the complete visible scope. The first mechanism makes dependencies schedulable; the second lets callbacks and backend adapters inspect the same lexical state that the host used to construct the invocation. [crates/ostadix-api/src/ir.rs, lines 760–991] [crates/ostadix-api/src/eval.rs, lines 2818–3118]

4.4 Persistent and ephemeral evaluators

```
python[0]^(  
  x = 40  
)_python[0]  
  
python[0]^(  
  __oval_result__ = x + 2  
)_python[0]
```

Both blocks share the same language, explicit environment ID, and process policy, so the process registry reuses one actor and Python state persists. Bare `python^(...)_python` blocks each receive a fresh ephemeral identity and are cleaned up after execution. Different numeric IDs isolate interpreter-local state; ambient `HostWorld` resources remain shared. [crates/ostadix-api/src/process.rs, lines 1480–2129]

Two explicit state domains

Root **Store** nodes commit to the caller's `O` scope, and nested stores belong to the nested evaluation scope. Persistent evaluator actors retain the backend's own process state under their environment identity. This separation gives `O` lexical state a graph-visible lifetime while allowing a backend such as Python to preserve its native runtime state across selected invocations.

independent HGRAPH resource lanes

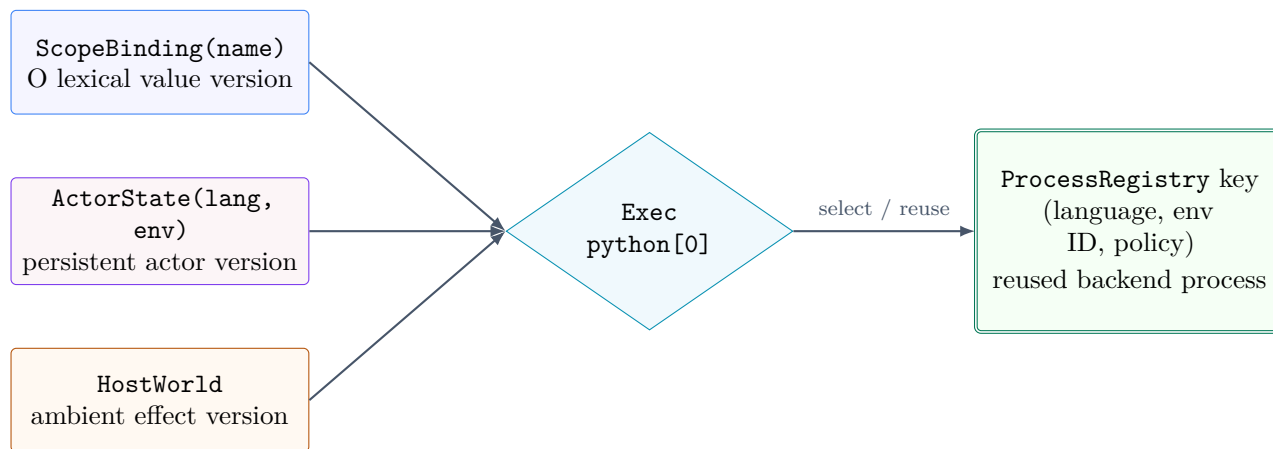


Figure 12: Lexical bindings, persistent actor state, and ambient host effects remain separate graph resources. **ActorState** uses canonical language and environment identity; concrete process reuse additionally includes the sandbox policy.

5 Hosted dynamic semantics

5.1 Evaluation order

The default public path strips a shebang, parses the document, lowers it, and executes the graph. `O_EXECUTOR=serial` selects the reference interpreter; `graph` is the default. The graph executor preserves required data, sequence, completion, and resource-state dependencies. [src/main.rs, lines 15–315] [crates/ostadix-api/src/eval.rs, lines 726–763]

At document level, evaluation returns the last non-null root result. Raw regions preserve backend source. A typed block:

1. evaluates structural children and loads visible bindings;
2. renders child values using the selected backend's splice renderer;
3. resolves an inline evaluator or an external backend actor;
4. sends source, environment identity, bindings, and policy;
5. decodes the backend result into OVALUE.

5.2 Quotation and meta-evaluation

`quote` returns an unevaluated `OExpr`. A Python shim can invoke `O.eval(expr)`, which reparses and lowers the quoted form under a cloned callback scope. `O.scope(map)` constructs an explicit detached scope, and the two-argument `O.eval(expr, scope)` form chooses it. [\[crates/ostadix-api/src/eval.rs, lines 2039–2087\]](#) [\[crates/ostadix-api/src/eval.rs, lines 2938–3118\]](#)

```
python[0]^(
  q = quote^(python^(6 * 7)_python)_quote
  __oval_result__ = O.eval(q)
)_python[0]
```

The callback path clones or explicitly selects an `O` scope, reparses the quoted expression, and sends its result back through the waiting actor protocol. Actor identity also provides a deadlock guard: an actor waiting on its own callback rejects recursive re-entry. Deferred shim forcing follows a single-level process path. [\[crates/ostadix-api/src/eval.rs, lines 1875–1987\]](#) [\[crates/ostadix-api/src/process.rs, lines 919–929\]](#)

5.3 Demand policies and requests

The evaluator exposes three demand policies:

Policy	Effect	Resolution rule
Eager	Resolve normal calls and blocks immediately.	The ordinary document policy.
Lazy	Construct a request or cache-safe deferred value.	<code>{lazy}</code> is accepted only for trusted inline renderers whose fingerprint covers the relevant inputs.
Autonomous	Buffer eligible requests for scheduler resolution.	Strict cache behavior can turn a missing result into an error after flush.

`{defer}` captures an evaluator request for explicit forcing with `now`; its request remains non-cacheable, and an unforced splice is a typed evaluation error. `{lazy}` attaches a cache-safety contract to trusted inline evaluation, so the runtime can force it on demand and reuse the fingerprinted result. [\[crates/ostadix-api/src/execution_contract.rs, lines 83–162\]](#) [\[crates/ostadix-api/src/execution_contract.rs, lines 218–285\]](#) [\[crates/ostadix-api/src/eval.rs, lines 1875–2022\]](#) [\[crates/ostadix-api/src/eval.rs, lines 2874–2924\]](#)

Autonomous resolution is a fingerprint-keyed, two-level scheduler. It first collects the transitive request closure and deduplicates the DAG, promotes results through an L1 memory cache and an L2 JSON disk

cache, then dispatches uncached ready nodes in topological waves. Default parallelism is the smaller of the available CPU count and eight, with an explicit caller override. The L2 path resolves through `XDG_CACHE_HOME`, then `HOME`, then the platform temporary directory. Each cache entry is written beside its final path and atomically renamed. A missing, unreadable, malformed, or unwritable cache entry is a soft cache event: execution proceeds and preserves the result contract. [\[crates/ostadix-api/src/scheduler.rs, lines 1–38\]](#) [\[crates/ostadix-api/src/scheduler.rs, lines 76–147\]](#) [\[crates/ostadix-api/src/scheduler.rs, lines 233–350\]](#)

The Nix request ladder is explicit:

$$\text{NixExpr} \xrightarrow{\text{instantiate}} \text{Derivation} \xrightarrow{\text{realise}} \text{StorePath} \xrightarrow{\text{activate}} \text{System}.$$

`dry_activate` constructs a preview request, `activate(path)` performs the ambient-authority activation, and `current_system` produces a referential system value. Separate request constructors make inspection and state-changing activation distinct in the value algebra and scheduler.

The perform boundary is concrete. `instantiate` wraps the source as `(let v = (BODY); in v.drvPath)`, runs `nix eval -raw -impure -expr`, validates the returned `/nix/store/*.drv` path, and asks `nix derivation show` for its output names, using `out` when output inspection fails. `realise` runs `nix build DRV^out -no-link -print-out-paths` and validates the printed store path. Activation executes the closure’s `bin/switch-to-configuration` with `switch` or `dry-activate` and supplies the chosen profile in `NIX_PROFILE`. [\[crates/ostadix-api/src/eval.rs, lines 3167–3340\]](#) [\[crates/ostadix-api/src/nix_ops.rs, lines 33–145\]](#) [\[crates/ostadix-api/src/nix_ops.rs, lines 176–258\]](#) [\[crates/ostadix-api/src/nixos_ops.rs, lines 29–101\]](#)

5.4 Coordination groups

Mode	Settlement	Current behavior
<code>batch</code>	Collect every outcome in member order; failures become structured error values.	Threadable work respects scheduler parallelism.
<code>all</code>	Require every member to succeed; otherwise fail the group.	Threadable work respects scheduler parallelism.
<code>any</code>	Return the first success; fail only when all fail.	Probe serial members in source order; if none succeeds, launch the complete threadable cohort.
<code>race</code>	Return the first settlement, success or failure.	The first serial member settles the group when one is present; otherwise race the complete threadable cohort.

Group constructors capture unevaluated arguments, preserving their computation topology until group resolution. Only Nix-family requests are classified as threadable in the group path; arbitrary shim evaluation remains serial. [\[crates/ostadix-api/src/eval.rs, lines 1540–1873\]](#)

Two concurrency planes

Graph-level parallelism is derived from value, completion, and resource dependencies, then admitted for verified-safe inline operations. Group-level concurrency is selected explicitly by `batch`, `all`, `any`, or `race` and currently dispatches the threadable Nix request family. One plane extracts parallel work from program structure; the other gives the program an explicit settlement topology.

6 The OValue semantic boundary

OVALUE is the cross-evaluator tagged algebra. It carries values, references, control objects, and error information across runtimes. [crates/ostadix-api/src/value.rs, lines 679–986]

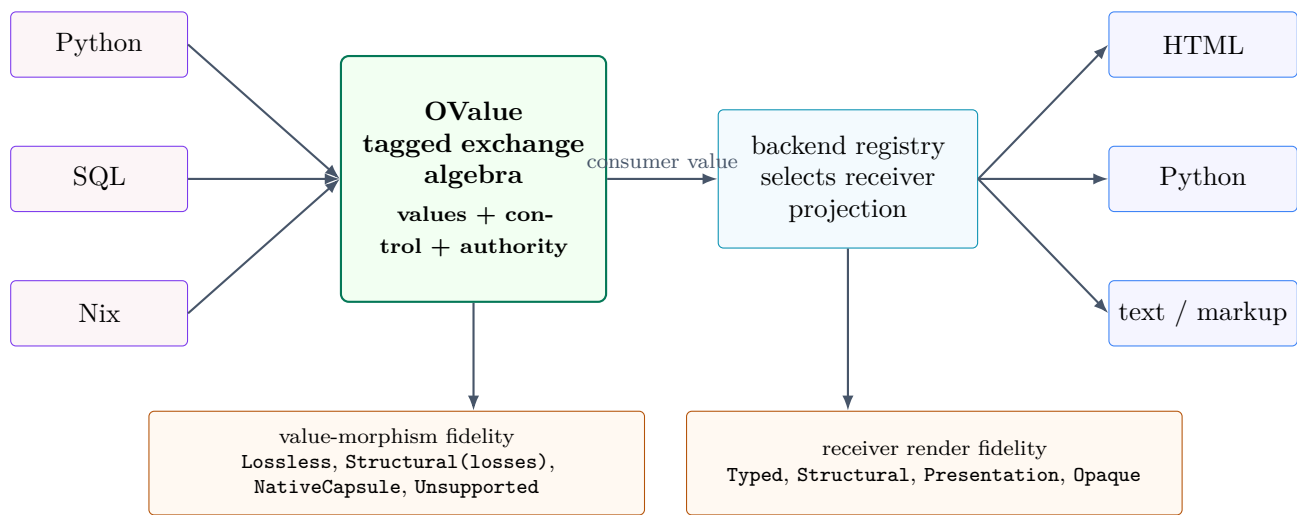


Figure 13: Every producer lifts into one inspectable algebra. The receiving backend then projects that value into its own source domain. Value conversion and source rendering retain distinct, composable fidelity judgments.

Family	Implemented variants
Scalars	Null, Bool, Number, Text, Char, HTML, store paths, and expressions.
Numbers	Arbitrary-precision integers, rationals, decimals including special values, binary floats, arbitrary-precision floats, and complex numbers.
Collections	List, Map, Seq, Object, entry-preserving map, Set, Symbol, and Keyword.
Rich payloads	Blob, Bytes, Graph, and Native capsule.
Execution state	Scope, Snapshot, Thunk, Request, Group, structured Error.
World references	NixExpr, Derivation, System, Capability.

The public subalgebras make collection shape, authority, snapshots, native rehydration, and runtime persistence explicit:

Subalgebra	Variants
SeqKind	List, Tuple, Vector.
SetKind	Ordered, Unordered.
CapabilityKind	File, MemoryRegion, Device, Clock, NetworkEndpoint, Process, Service, SystemActivation, BackendExecution.
SnapshotKind	System, Service, Filesystem, Device.
RequestKind	Instantiate, Realise, Eval, Activate.
NativeCodecSafety	DataOnly, SourceBacked, UnsafeOpaque, LiveHandle.
NativeBoundary	Pure, Referential, Effectful.

Subalgebra	Variants
RehydratePolicy	Portable, SameBackend, SameProcess, Never.
RuntimeBoundary	Pure, Referential, Effectful.
Fidelity	Lossless, Structural, NativeCapsule, Unsupported.
AnnotationKind	TypeTag, NumericPrecision, NumericExactness, Encoding, Ordering, Identity, Constraint, Capability, Presentation, and an extensible BackendSpecific label.

6.1 Authorable tagged OValue schema

Every JSON or CBOR OValue is selected by its top-level **t** field. Fields named as values, items, members, bindings, metadata, or state contain OValues recursively.

t tag	Fields and nested vocabulary
null; bool number	Null consists exactly of the t tag. Bool adds Boolean v . v.kind is <code>int{v}</code> , <code>rational{num,den}</code> , <code>decimal{coeff,exp10,special?}</code> , <code>binary_float{format,bits}</code> , <code>big_float{mantissa,exp2,precision?,special?}</code> , or <code>complex{re,im}</code> . A question mark identifies an optional or nullable field. Canonical big integers are decimal strings; formats are <code>f32 f64</code> ; specials are <code>nan</code> , <code>pos_inf</code> , <code>neg_inf</code> , <code>pos_zero</code> , and <code>neg_zero</code> .
text; char; html; store_path; expr list; map seq; object; entries_map; set	Text has <code>v{utf8,encoding?}</code> ; char has <code>scalar</code> ; HTML has v ; store path has <code>path</code> ; expression has <code>src</code> . List v is an array; map v is a string-keyed object. Seq has <code>kind=list tuple vector</code> and <code>items</code> ; object has string <code>fields</code> ; entries-map has an <code>entries</code> array of key/value pairs; set has <code>kind=ordered unordered</code> and <code>items</code> .
symbol; keyword; scope	Symbol and keyword v each contain optional or nullable <code>namespace</code> and required <code>name</code> ; scope has string-keyed <code>bindings</code> .
blob; bytes	Blob has <code>base64 v</code> and <code>mime</code> . Bytes has <code>v{bytes,media_type}</code> , with an octet array and nullable media type.
graph	root is a node ID; nodes contain <code>kind=value</code> with <code>value</code> , or <code>kind=ref</code> with <code>target</code> .
native	v contains <code>lang</code> , nullable <code>implementation/version</code> , <code>type_name</code> , <code>identity{stable?,live?}</code> , <code>codec</code> , optional or nullable byte <code>payload</code> , <code>boundary</code> , <code>safety</code> , <code>capabilities</code> , <code>metadata</code> , and <code>rehydrate</code> . Boundary is <code>pure referential effectful</code> ; safety is <code>data_only source_backed unsafe_opaque live_handle</code> ; rehydration is <code>portable same_backend same_process never</code> .
nix_expr; derivation	Nix expression has <code>body</code> , <code>deps</code> , and <code>fingerprint</code> ; derivation has <code>drv_path</code> , <code>outputs</code> , and <code>deps</code> .
request; system	Request has <code>kind</code> , <code>source</code> , and <code>fingerprint</code> . Kind is <code>instantiate</code> , <code>realise</code> , an eval object with <code>lang</code> , <code>env_id</code> , <code>cacheable</code> , nullable <code>authority</code> , and <code>permissions</code> , or an activate object with <code>profile</code> , <code>dry_run</code> , and nullable <code>authority</code> . System has <code>profile_path</code> .

t tag	Fields and nested vocabulary
<code>capability;</code> <code>snapshot</code>	Capability has <code>kind</code> , opaque <code>identity</code> , and descriptive <code>metadata</code> . Snapshot has <code>kind=system service filesystem device</code> , <code>identity</code> , and <code>state</code> .
<code>thunk; group; error</code>	Thunk has <code>body</code> , <code>deps</code> , and <code>fingerprint</code> . Group has <code>mode=batch all any race</code> , <code>members</code> , and <code>fingerprint</code> . Error has <code>msg</code> .
Legacy input tags	<code>int{v}</code> , <code>float{v}</code> , and <code>str{v}</code> are accepted and normalize to canonical number or text.

Capability kinds are `file`, `memory_region`, `device`, `clock`, `network_endpoint`, `process`, `service`, `system_activation`, and `backend_execution`. [crates/ostadix-api/src/value.rs, lines 94–663] [crates/ostadix-api/src/value.rs, lines 679–1246] [crates/ostadix-api/src/value.rs, lines 1257–1475]

6.2 Fidelity

Conversions carry one of four fidelity states: `Lossless`, an accumulated set of structural losses, `NativeCapsule`, or `Unsupported`. Composition terminates at a fixpoint and preserves the weakest accumulated guarantee. [crates/ostadix-api/src/value.rs, lines 155–547]

`BackendMorphismV1` defines bounded Python, JavaScript, and Rust crossing profiles with explicit input projection, output injection, rejection kinds, two fidelity legs, and a checked lossless round-trip law. Catalog V5 binds each profile to its canonical backend entry. The HGraph solver exposes the resulting assessment beside its compatibility fidelity result in shadow mode, making profile divergences inspectable while the established admission, rendering, and dispatch paths retain their current authority. [crates/ostadix-api/src/backend_morphism.rs, lines 1–7] [crates/ostadix-api/src/backend_morphism.rs, lines 19–40] [crates/ostadix-api/src/backend_morphism.rs, lines 126–271] [crates/ostadix-api/src/backend_morphism.rs, lines 365–404] [crates/ostadix-api/src/hgraph/solve.rs, lines 874–881]

6.3 Representation-sensitive identity

- Rational identity preserves the supplied numerator, denominator, and sign representation after rejecting a zero denominator.
- Unordered sets sort canonical item encodings; ordered sets preserve declared order; both preserve encoded multiplicity.
- System identity is referential and based on profile path.
- Snapshot identity uses kind and referential identity, leaving changing snapshot state outside the stable identity key.
- Schema encoding preserves descriptors for capabilities, scopes, native handles, and host effects; their live authority remains bound to the runtime that owns them.

[crates/ostadix-api/src/value.rs, lines 1507–1518] [crates/ostadix-api/src/value.rs, lines 1785–1858] [crates/ostadix-api/src/value.rs, lines 1954–2254] [crates/ostadix-api/src/value.rs, lines 2529–2795]

6.4 Wire protocol

Backend commands and responses use four-byte big-endian length-prefixed CBOR with a 128 MiB frame limit. Map encoding is canonicalized. Decoding rejects trailing bytes, indefinite encodings, malformed forms, non-text map keys, and non-finite encodings outside the defined value schema. [crates/ostadix-api/src/wire.rs, lines 1–105] [crates/ostadix-api/src/canonical_cbor.rs, lines 1–23] [crates/ostadix-api/src/canonical_cbor.rs, lines 49–85] [crates/ostadix-api/src/canonical_cbor.rs, lines 157–220]

The Rust serializer emits canonical **number** and **text** frames. Compatibility shims can emit legacy **int**, **float**, and **str** frames; Rust decoding normalizes them into `ONumber` and `OText` before the value enters the hosted algebra.

The complete shim-extension vocabulary is:

Direction	Tagged CBOR body
Runtime to shim	<code>cmd=exec</code> with code and a string-keyed <code>bindings</code> map of <code>OValues</code> .
Runtime to shim	Fieldless <code>cmd=cleanup</code> and <code>cmd=ping</code> .
Runtime to shim	<code>cmd=eval_result</code> with value.
Shim to runtime	<code>status=ok</code> with value; or <code>status=err</code> with <code>message</code> .
Shim to runtime	<code>status=eval_request</code> with <code>src</code> and optional <code>scope</code> . The runtime evaluates that <code>O</code> source, returns <code>eval_result</code> , and the paused shim resumes until <code>ok</code> or <code>err</code> .

[crates/ostadix-api/src/value.rs, lines 3318–3380]

7 Backends, actors, and authority

7.1 Static registry

The compile-time Rust registry binds each recognized language tag to its renderer, execution mode, cache-safety classification, and required authority. It declares 30 canonical tags plus six aliases. Seven structural/value handlers execute inline or as builtins; external evaluation is mediated by native adapters and bundled compatibility shims. A single registry entry therefore connects surface syntax to planning metadata, state support, morphism profile, runtime requirements, and runtime dispatch. [crates/ostadix-api/src/backend_catalog.rs, lines 623–685] [crates/ostadix-api/src/backend_catalog.rs, lines 841–921] [crates/ostadix-api/src/backend_catalog.inc.rs, lines 161–590] [crates/ostadix-api/src/shims.rs, lines 7–182]

Class	Registered canonical tags
Structural/value	<code>0</code> , <code>quote</code> , <code>nix_expr</code> , <code>html</code> , <code>markdown</code> , <code>latex</code> , <code>text</code> .
Nix and systems tests	<code>nix</code> , <code>nix_store</code> , <code>nixos_test</code> , <code>ubuntu_vm</code> .
Data and scripting	<code>sql</code> , <code>python</code> , <code>bash</code> , <code>shell</code> , <code>javascript</code> , <code>ruby</code> , <code>matlab</code> , <code>mathematica</code> .
Compiled and functional	<code>haskell</code> , <code>ocaml</code> , <code>webassembly</code> , <code>rust</code> , <code>racket</code> , <code>csharp</code> , <code>c</code> , <code>cpp</code> , <code>lisp</code> , <code>common_lisp</code> , <code>java</code> .

The six aliases are exact parser spellings: `o` maps to `0`; `py` maps to `python`; `md` maps to `markdown`; `tex` maps to `latex`; and `plain` maps to `text`; `ubuntu` maps to `ubuntu_vm`. Block attributes attach execution

metadata directly to the evaluator-indexed form. The implemented attribute surface is `cap=name`, the four authority flags `fs_read`, `fs_write`, `network`, and `process`, `lazy`, `defer`, `effects=pure|unknown`, `reads=...`, `writes=...`, and `serial=host`. Effect-resource lists use `+` or `;` as separators. The resource spellings are:

<code>project:path</code>	<code>host:path</code>	<code>env:NAME</code>
<code>scope:name</code>	<code>network:endpoint</code>	<code>service:name</code>
<code>actor:language[index]</code>	<code>evaluator</code>	<code>stdio</code>
<code>hostworld</code>		

These spellings let source describe graph resources in the same vocabulary used by the planner. [crates/ostadix-api/src/parser.rs, lines 590–738] [crates/ostadix-api/src/effects.rs, lines 1–434]

The repeatable backend-grant syntax is `NAME=LANG[:RIGHT,RIGHT]`. `NAME` begins with an ASCII letter or underscore and continues with ASCII letters, digits, or underscores. `LANG` is nonempty; known aliases canonicalize, and `*` creates a wildcard. Parser dispatch still requires a registered source tag. Rights are `fs_read`, `fs_write`, `network`, and `process`; commas separate them and duplicates collapse. An omitted or empty rights list produces a zero-rights bearer. A block that declares one or more authority flags uses `cap=NAME` as its first bearer candidate. A match must contain every declared right; an absent or insufficient binding follows the evaluator’s default-authority path. The release CLI gate executed `runner=python:process`. [crates/ostadix-api/src/eval.rs, lines 1217–1227] [tests/test_cli.sh, lines 888–898]

At invocation time, the adapter resolves its host executable and converts the subset of `OVALUE` accepted by that runtime. This keeps language recognition deterministic while allowing the executable environment to be selected by the host installation.

7.2 Actor process model

Production external evaluation self-spawns the current executable in hidden `-o-backend` mode. The process registry key is

(language, environment ID, sandbox policy).

Persistent keys reuse a process; ephemeral keys are removed after evaluation. Most backends are Rust adapters that invoke local executables. Python and `nixos_test` use legacy Python proxy shims. Python retains this path for the host callback protocol used by `O.eval`. [crates/ostadix-api/src/process.rs, lines 554–609] [crates/ostadix-api/src/backend.rs, lines 77–743]

Backend stdout is decoded in a defined fallback order: tagged or ordinary JSON, integer/big integer, float, then text. SQL additionally collapses a single-row, single-column result. Environment conversion is narrower than the full `OVALUE` universe. [crates/ostadix-api/src/backend.rs, lines 1229–1503]

7.3 Hosted authority model

The source contains random 256-bit bearer identities, typed backend rights (`fs_read`, `fs_write`, `network`, `process`), broker checks, revocation, and policy-partitioned process reuse. [crates/ostadix-api/src/capability.rs, lines 13–188]

The ordinary CLI executes backends with the ambient authority of the `O` process. Its launch path realizes that model in five steps:

1. evaluator startup mints a wildcard bearer with all four rights;

2. source `cap=` resolution falls back to that wildcard;
3. normal shim launch policies expand to all rights;
4. the production Python proxy runs under the outer process policy;
5. the macOS profile begins with `allow default` and derives denials from rights omitted by a narrowed embedding policy.

[crates/ostadix-api/src/eval.rs, lines 766–796] [crates/ostadix-api/src/eval.rs, lines 1230–1304] [crates/ostadix-api/src/process.rs, lines 167–241] [crates/ostadix-api/src/process.rs, lines 637–683]

Authority is explicit runtime state

Hosted capability machinery supplies random bearer identity, typed rights, revocation, policy-partitioned actor reuse, deferred-request revalidation, and embedding hooks. The public CLI chooses the ambient policy, maximizing backend compatibility and execution capacity. An embedding can supply a narrowed process policy and reuse the same identity and revalidation path.

8 OIR, plans, effects, and HGraph execution

8.1 OIR is orchestration IR

OIR has five node forms:

Text Load Store Invoke Exec.

These forms model the complete hosted orchestration surface: source fragments, lexical dataflow, binding publication, builtin invocation, and evaluator execution. Lowering from the parser AST is structural; validation then turns that representation into an executable plan. The compact vocabulary makes every hosted construct projectable into the same graph ontology. [crates/ostadix-api/src/ir.rs, lines 1–21]
[crates/ostadix-api/src/ir.rs, lines 78–171]

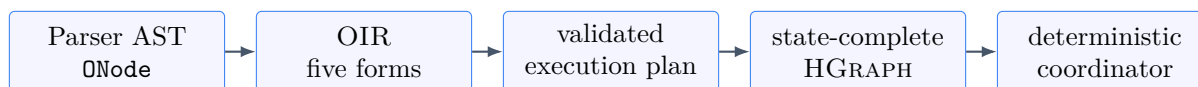


Figure 14: Every ordinary hosted execution is mediated by the same lowering seam; serial execution remains a differential oracle.

8.2 Execution plan

Plan node IDs are preorder-stable. Edges encode child-before-parent structure, sibling sequencing, binding dataflow, capability dependencies, and the full visible scope sent to shims. Each executable node freezes the registry-derived `BackendInterface`; execution rejects an interface that no longer matches the registry. [crates/ostadix-api/src/ir.rs, lines 540–657] [crates/ostadix-api/src/ir.rs, lines 760–991] [crates/ostadix-api/src/eval.rs, lines 2704–2891]

8.3 State-complete HGraph

Each ordinary plan node maps to one value node. Every executable hyperedge also consumes required completion tokens and versioned resource states, then atomically produces:

1. one ordinary value;
2. one completion token;
3. successor versions for every accessed resource.

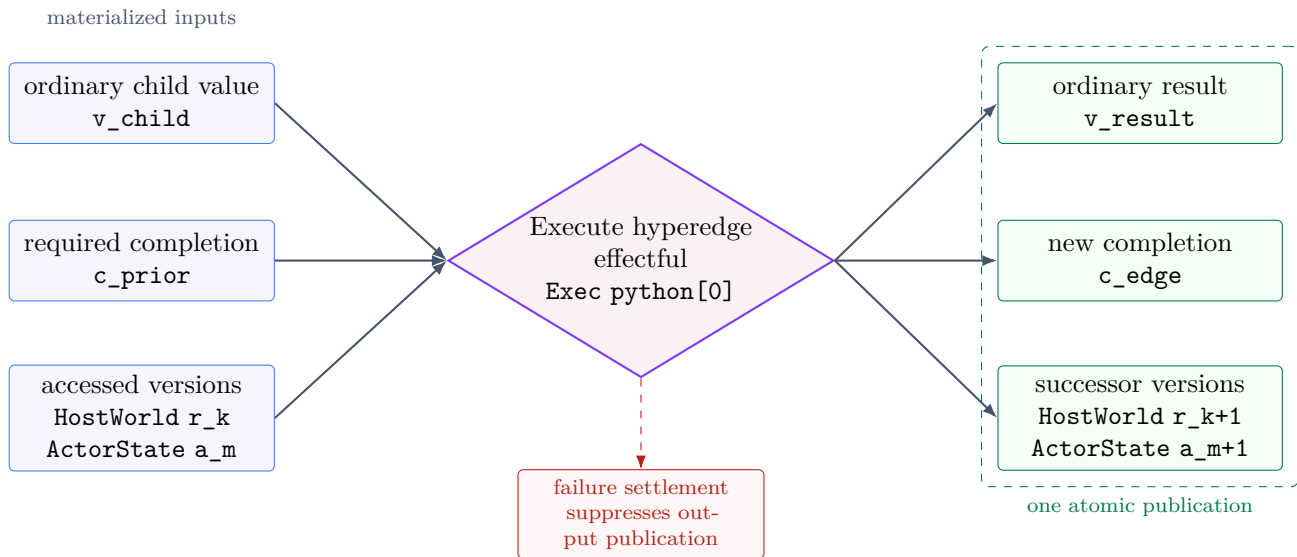


Figure 15: An executable HGRAPH hyperedge consumes value, completion, and resource state together. This persistent evaluator example advances `HostWorld` and `ActorState(python[0])`. Success publishes the result, completion token, and every successor resource version as one graph transition; failure withholds those graph outputs.

[crates/ostadix-api/src/hgraph/from_oir.rs, lines 35–291]

Validation checks unique producers, complete outputs, exact source/plan provenance, sequence preservation, contiguous resource chains, and executable acyclicity. The ready schedule is derived from unresolved inputs and their producers with stable wave and ordinal ordering. [crates/ostadix-api/src/hgraph/graph.rs, lines 368–515] [crates/ostadix-api/src/hgraph/graph.rs, lines 1454–1908] [crates/ostadix-api/src/hgraph/schedule.rs, lines 10–173]

8.4 Effects and explicit resource state

Resources include `HostWorld`, evaluator state, scope/bindings, project and host paths, environment variables, stdio, network endpoints, services, and actor state. Work with the general effect classification receives a `HostWorld` dependency, serializing it against other world-mutating operations. Source declarations add restrictions and dependencies; verified purity is assigned by the trusted inline registry classification. [crates/ostadix-api/src/effects.rs, lines 46–636]

Only attribute-free inline text, HTML, Markdown, and LaTeX work is treated as verified pure and worker-eligible. Effectful ready work executes one operation at a time in deterministic ordinal order. Successful outputs publish atomically; the root scope commits deterministically. [crates/ostadix-api/src/effects.rs, lines 776–843] [crates/ostadix-api/src/executor/coordinator.rs, lines 103–1094]

Why the state-complete graph matters

The graph is richer than a syntax tree because it represents value, completion, and mutable-world dependencies separately. Its present execution policy preserves observable equivalence and deterministic settlement while parallelizing independently verified inline work. The same resource model identifies the exact admission point for additional concurrent adapters: they must declare effects precisely enough to remove unnecessary `HostWorld` dependencies.

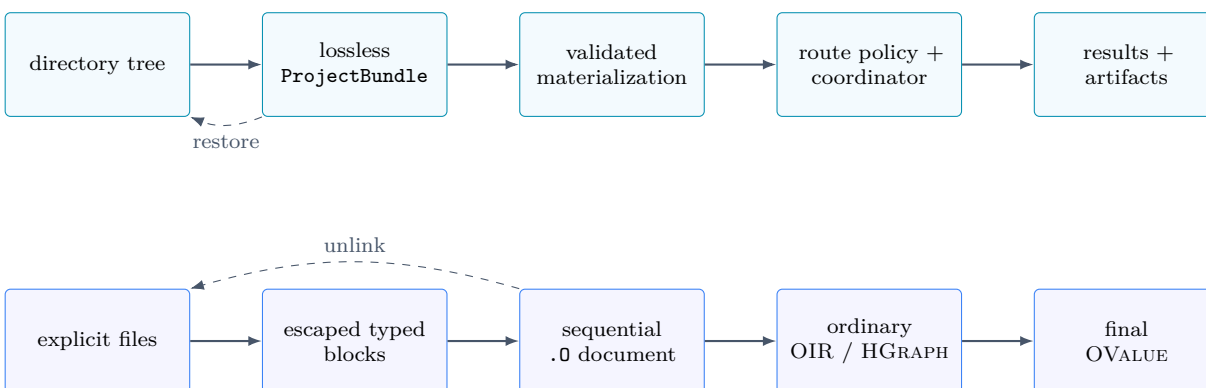
9 Projects, linking, and route policies

9.1 Project lifting and literal linking

`o-link` has two materially different composition lanes:

- A bare single directory follows the literal lane and runs the linked result. Explicit `-literal` emits the literal linked document; `-literal -run` executes it.
- Explicit `-project` captures a deterministic route-preserving project bundle. `-project -run` selects and executes a route through the project coordinator.

Project lane. `--project` lifts; `--project --run --route NAME` executes; a lifted `.O` reloads the project representation.



Literal lane. A bare directory links and runs; `--literal` links; `--literal --run` executes.

Figure 16: `o-link` preserves two composition semantics. Explicit project lift captures a reversible tree and selects whole-project routes; literal linking constructs one hosted program and a bare single directory runs that program by default.

[src/bin/olink.rs, lines 183–565] [src/bin/olink.rs, lines 841–1029]

9.2 Compiler-emitted Project HGraph from an o-linked codebase

The tracked fixture in `tests/fixtures/project_hgraph/` is a complete project codebase with one input file, a manifest, one prerequisite route, two implementation routes, declared inputs and outputs, an environment guard, and a `verify_equivalent` route set named `main`. I lifted that directory through the project form of `o-link`, asked `olangc -target dot` to render the selected route set, and compared the result with direct directory planning.

```

mkdir -p output/current-master
./target/release/o-link tests/fixtures/project_hgraph --project \
  -o output/current-master/o-linked-project.0

./target/release/olangc output/current-master/o-linked-project.0 \
  --target dot --route main --shim-dir backends \
  > output/current-master/o-linked-codebase-hgraph.dot

./target/release/olangc tests/fixtures/project_hgraph \
  --target dot --route main --shim-dir backends \
  > output/current-master/project-hgraph-direct.dot

cmp output/current-master/o-linked-codebase-hgraph.dot \
  output/current-master/project-hgraph-direct.dot

cp output/current-master/o-linked-codebase-hgraph.dot \
  docs/figures/o-linked-codebase-hgraph.dot
dot -Tsvg output/current-master/o-linked-codebase-hgraph.dot \
  -o docs/figures/o-linked-codebase-hgraph.svg
dot -Tpdf output/current-master/o-linked-codebase-hgraph.dot \
  -o docs/figures/o-linked-codebase-hgraph.pdf

```

The lifted artifact is 7,501 bytes with SHA-256 a6d97fd5b95b829c3e3a754722767960f2963e0644ba8c89da7c15d48d922bb1. The compiler-emitted DOT is 13,065 bytes with SHA-256 bf16748f53a377e950a67b9c27d48ff260dcc987a69d86c5f29ca6289df7f00b. The direct-directory DOT and lifted-artifact DOT are byte-identical. The SVG is 61,756 bytes with SHA-256 44d7d33abf7d34ed73e3f4f37ebee77db e5545f52cfc0077c64c021015310d01. The included vector PDF is 27,210 bytes with SHA-256 75c7d219d208a2afddc51a5c9c682c506d26cd9729145828019f88f1c6a8e90e.

[tests/fixtures/project_hgraph/olang.project.toml, lines 1–38] [src/bin/olink.rs, lines 713–749] [src/bin/olangc.rs, lines 556–580] [crates/ostadix-api/src/project/plan.rs, lines 1–25] [crates/ostadix-api/src/project/plan.rs, lines 46–87] [crates/ostadix-api/src/project/plan.rs, lines 91–140]

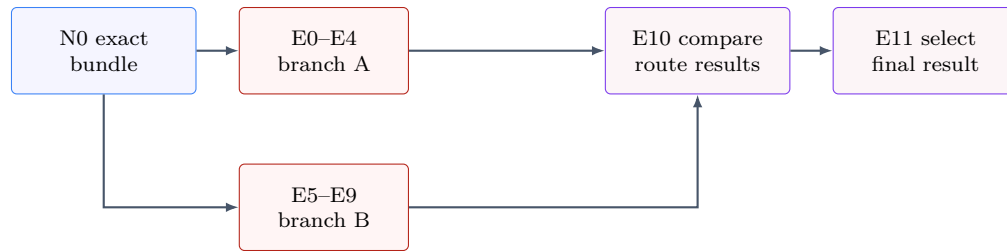
How to read the emitted graph

The figure declares 53 semantic vertices and 62 directed edges. Twelve rose **Execute** boxes are project operations. The other 41 vertices are values, completion tokens, and versioned resources. Read it in seven passes:

1. Begin at **Value** N0. It identifies the exact project bundle consumed by both alternative branches.
2. Find **Execute** E0 through **Execute** E4. This is the **impl-a** branch: materialize the project, build and run **prepare**, then build and run **impl-a**.
3. Find **Execute** E5 through **Execute** E9. This is the independently materialized **impl-b** branch with its own **prepare** prerequisite and **impl-b** execution.
4. Follow completion diamonds from each build operation into its run operation. A completion edge expresses successful prerequisite settlement; a value edge carries an ordinary result.
5. Follow cyan resource nodes. **HostWorld** advances from @0 through @6; project input, build, and output resources advance through the operations that read or write them.
6. Find **Execute** E10. It compares the two route results required by **verify_equivalent** after both alternatives have completed.

7. Finish at **Execute E11**. It selects the route-set result from the comparison outcome and publishes the final value and completion.

The graph should be read by dependency, then by page position. Arrow labels name the port through which a node participates: `input:value`, `input:completion`, `input:resource-state`, `result`, `completion`, and `resource-state`. Each `P# ordinal #` label binds an execute box to its project plan operation and gives deterministic ordering among simultaneously ready operations. A resource version belongs to the state after the operation that produced it. A completion token belongs to the terminal settlement of its producing operation.



each branch: materialize, build prepare, run prepare, build implementation, run implementation

Figure 17: Reading map for the current compiler-emitted project graph. The two source-closed alternative branches converge through comparison and final selection.

The panorama preserves the exact compiler artifact. The four detail figures that follow it enlarge overlapping horizontal windows from that same vector PDF. Read them from left to right. The overlap carries crossing arrows into the next window, so a port that leaves one page can be found again at the matching edge of the following page.

Topology index

Use this panorama to see branch shape and convergence. Read node and port labels in the four enlarged panels that follow.

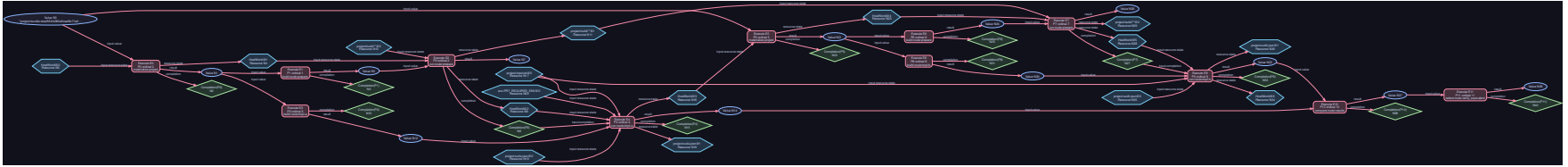


Figure 18: The exact directed Project HGraph emitted by `olangc -target dot -route main` from the o-linked `tests/fixtures/project_hgraph` codebase. The graph contains 12 executable operations, 41 value, completion, and resource vertices, and 62 directed dependency edges.

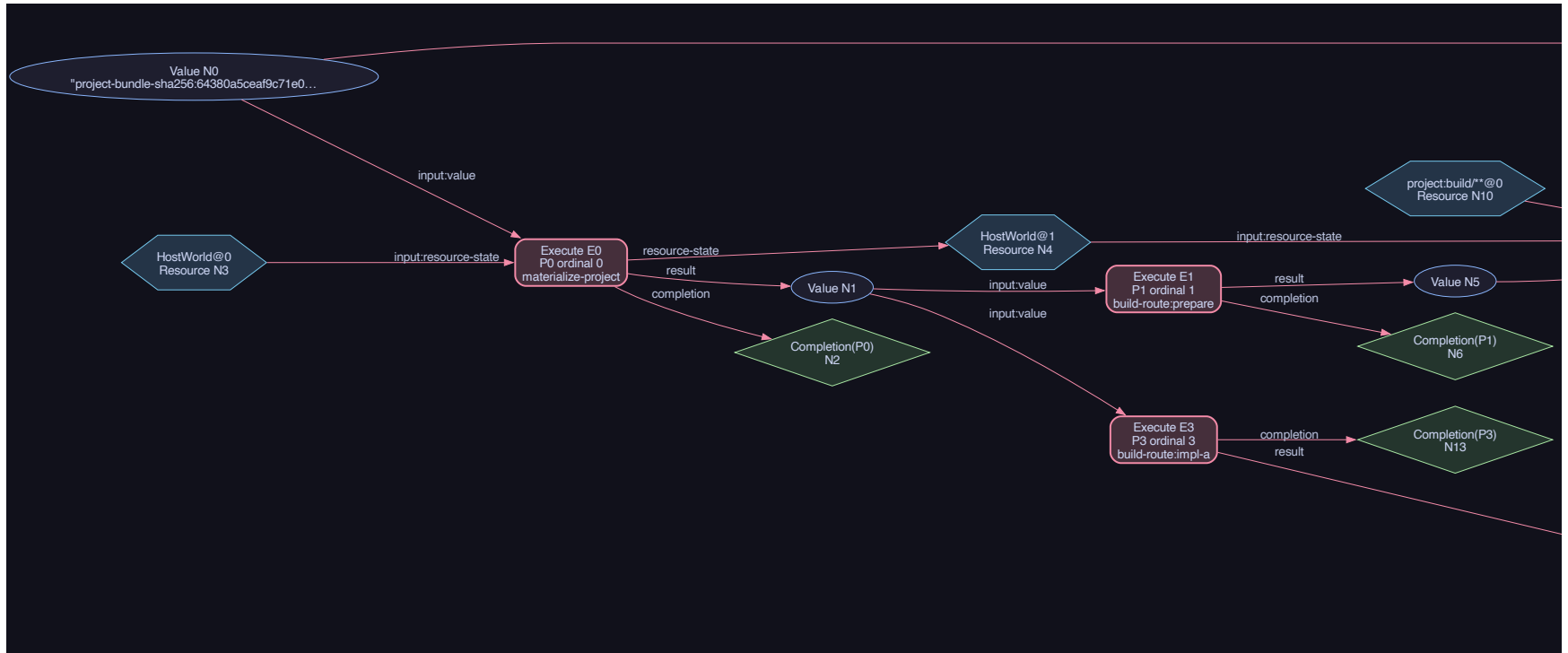


Figure 19: Project HGraph detail 1 of 4. Begin at the exact project identity, then follow materialization and the opening operations of the `impl-a` branch.

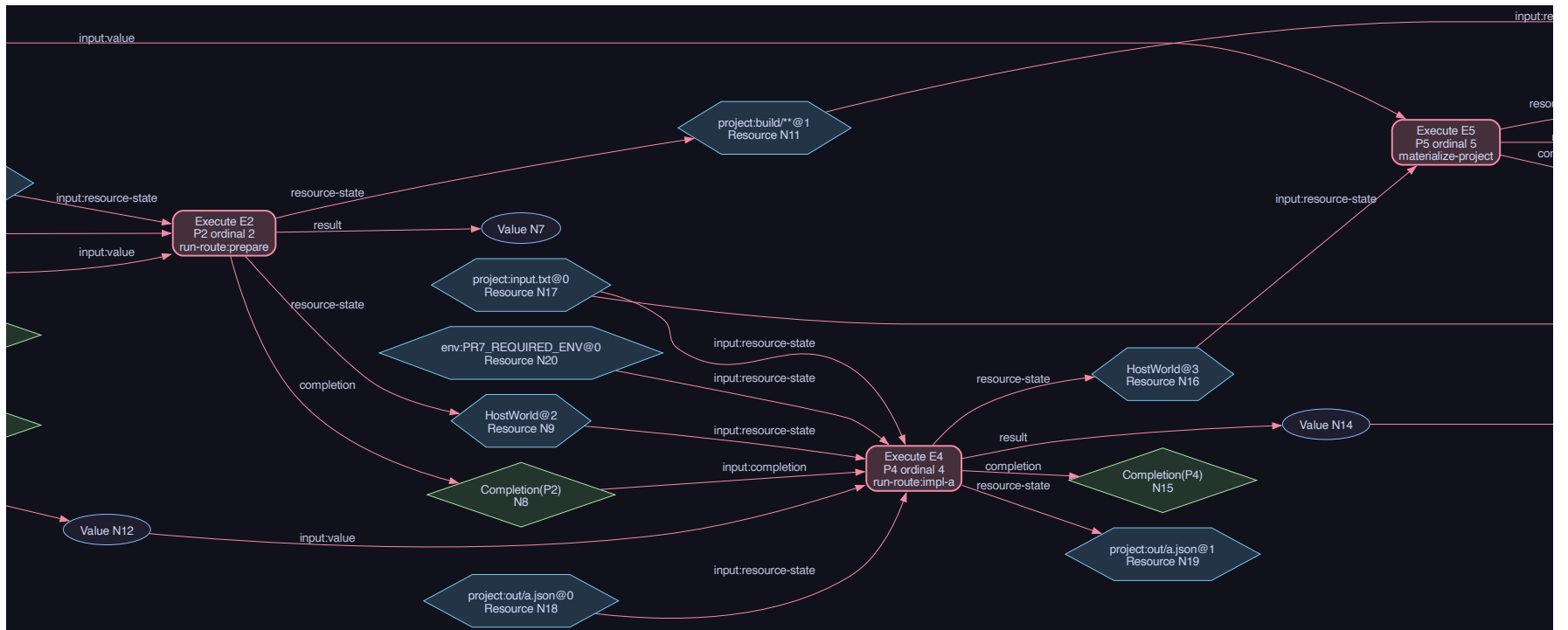


Figure 20: Project HGraph detail 2 of 4. Complete the `impl-a` route, then locate the independently materialized opening of the `impl-b` branch.

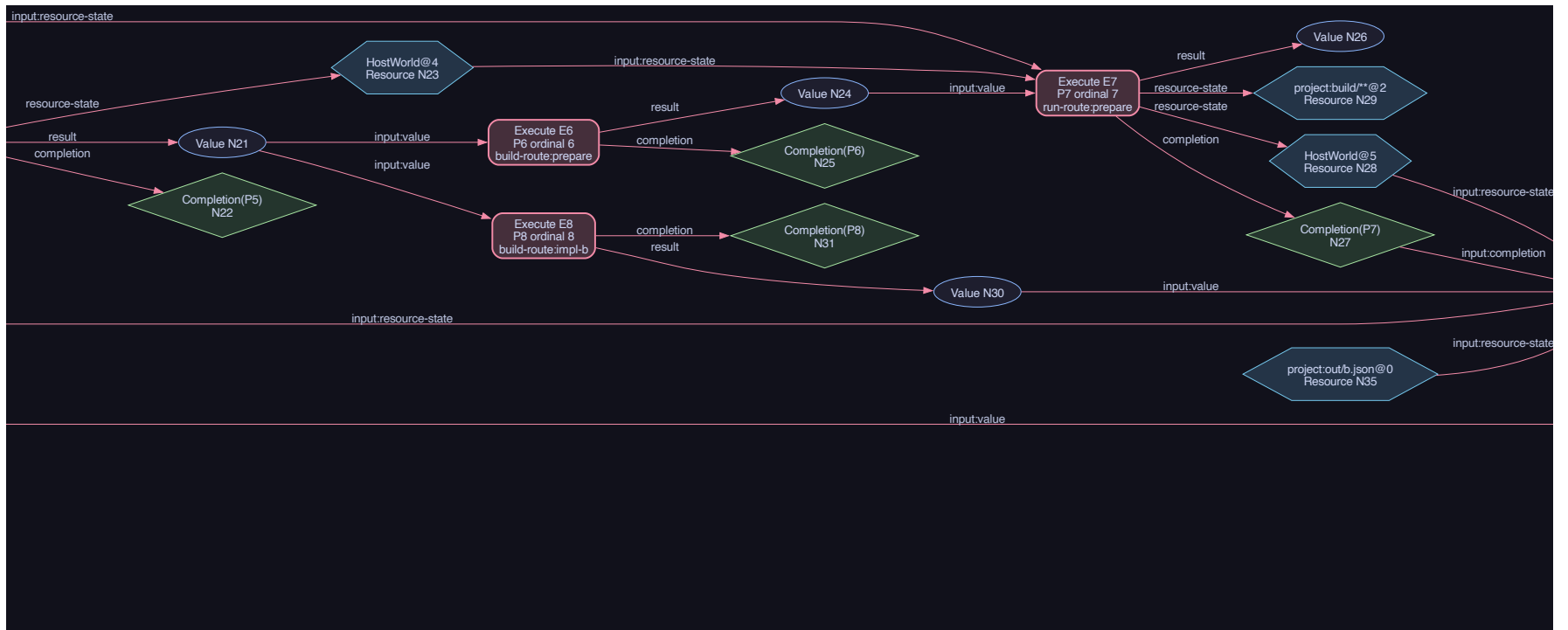


Figure 21: Project HGraph detail 3 of 4. Follow the build and run operations of `impl-b`, including their completion and resource-state ports.

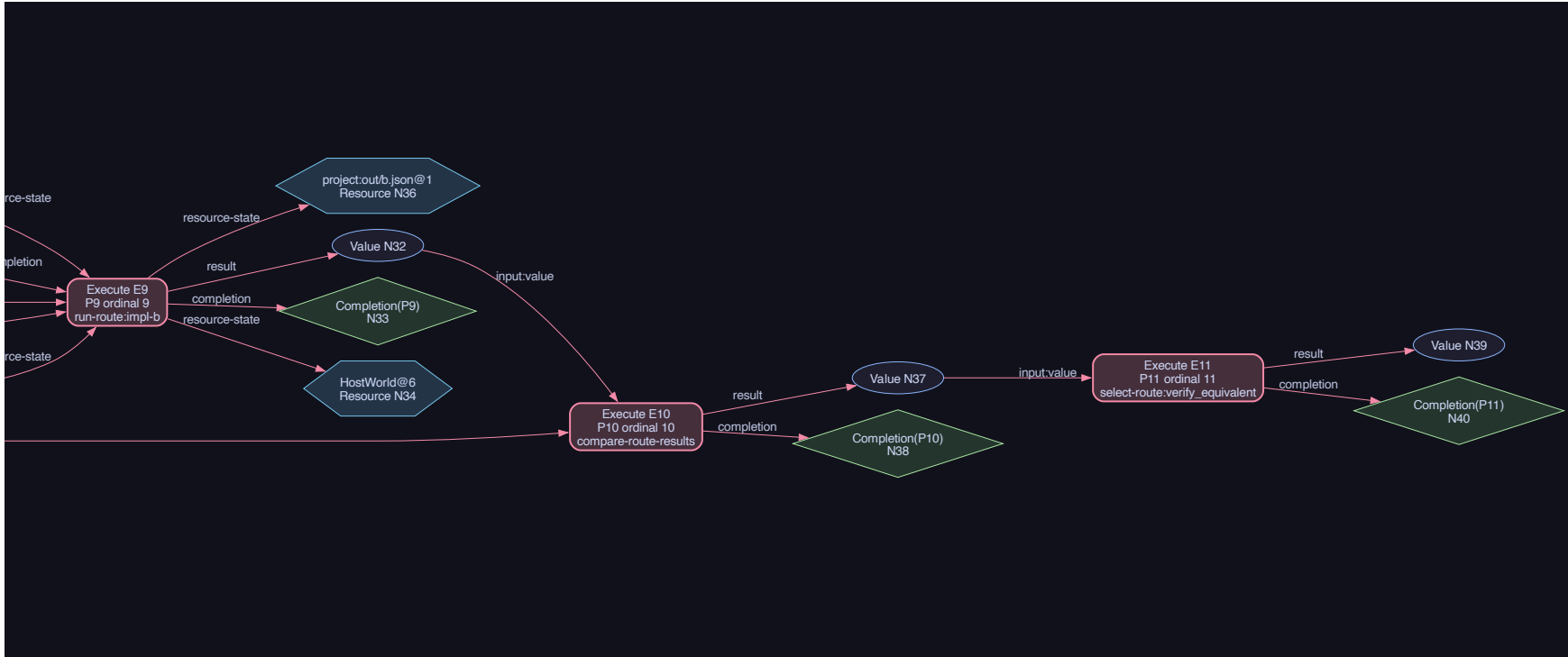


Figure 22: Project HGraph detail 4 of 4. Settle the second branch, compare both route results at E10, and publish the selected route-set result at E11.

The diagram exposes why project planning is a language operation. Directory capture, prerequisite closure, alternative isolation, input and output resources, equivalence comparison, and selection all become typed graph operations. The exact-equality check shows that lifting preserves the project semantics consumed by the compiler.

9.3 Bundle model

`ProjectFile` preserves relative path, raw bytes, Unix mode, symlink metadata, evaluator hint, role, and content hash. A project carries route specifications, guards, provenance, effects, and policy. Traversal is deterministic and skips generated/build/cache subtrees. Materialization validates relative paths, ancestor symlinks, and in-root symlink targets. [crates/ostadix-api/src/project/model.rs, lines 13–290] [crates/ostadix-api/src/project/bundle.rs, lines 28–137] [crates/ostadix-api/src/project/materialize.rs, lines 62–179]

The bundle fingerprint covers artifact identity through paths, content hashes, modes, and symlink flags. Route metadata remains a separately selected execution policy. On Unix, materialization reproduces modes and symlinks from the captured model. [crates/ostadix-api/src/project/bundle.rs, lines 123–137] [crates/ostadix-api/src/project/materialize.rs, lines 195–223]

Six file roles classify the captured tree: `Source`, `Asset`, `Manifest`, `GeneratedRoute`, `Metadata`, `Entrypoint`, and `Other`. Route discovery proceeds in a fixed ecosystem order covering Rust, Python, JavaScript, shell, C-family, Java, .NET, Haskell, OCaml, Nix, and generic entrypoints. An `olang.project.toml` manifest can state exact routes and route sets while retaining the captured files as the artifact of record. [crates/ostadix-api/src/project/discover.rs, lines 1–132] [crates/ostadix-api/src/project/manifest.rs, lines 1–378]

9.4 Routes

The native project coordinator implements `explicit`, `default`, `fallback`, `any_success`, `race_success`, `race_settle`, `all`, `verify_equivalent`, and `benchmark_and_select`. Alternatives run as subprocess routes with isolated or shared workspaces as required; both race policies cooperatively cancel losing route processes. [crates/ostadix-api/src/project/model.rs, lines 310–402] [crates/ostadix-api/src/project/runtime.rs, lines 2735–3240]

The project coordinator owns whole-tree materialization, route selection, subprocess lifecycle, result settlement, and workspace sharing. Ordinary OIR/HGRAPH execution owns expression-level evaluator composition. This division gives project routes enough context to coordinate complete build and run alternatives while preserving the hosted language’s value graph. [crates/ostadix-api/src/project/manifest.rs, lines 40–167] [crates/ostadix-api/src/project/runtime.rs, lines 695–704]

Route model	Variants	Recorded execution facts
Kind	Interpreter command, compiled binary, build target, package entrypoint, shell task, O evaluator, composite.	Identity, command or evaluator, entrypoint, working directory, arguments, environment, prerequisites, inputs, outputs, priority, and default status.
Codec	Text, JSON, bytes.	Defines stdout interpretation and the optional structured JSON result.
Guard	Operating system, command availability, environment variable.	Controls candidate eligibility before launch.

Route model	Variants	Recorded execution facts
Provenance	Manifest, discovery, CLI override.	Records where executable policy entered the bundle.
Effects	Pure or unknown, plus read and write resources.	Connects project declarations to scheduling and workspace decisions.
Result	One settled route execution.	Route ID, exit status, stdout and stderr bytes, optional JSON, artifacts, duration, command provenance, and workspace provenance.

9.4.1 Authoring project routes

`olang.project.toml` exposes the complete manifest route vocabulary and route-set policy as an authorable format. Manifest routes replace discovered routes with the same ID, and route sets replace sets with the same `provides` key. The reader tolerates additional fields so future metadata can remain attached to a bundle.

Section	Fields
[project]	Optional name and default_route.
[[routes]]	Required id; optional label, command, kind, evaluator, entrypoint, cwd, args, env, depends_on, inputs, outputs, provides, result_codec, priority, default, guards, and pure.
[routes.guards]	Optional os, requires_command, and requires_env gates on the preceding route.
[[route_sets]]	Required provides and alternatives; optional policy: explicit:ID, default, fallback, any_success, race_success, race_settle, all, verify_equivalent, or benchmark_and_select.

The repeatable `-route-decl` override uses semicolon-separated `key=value` fields. `id` and `cmd` are required; the remaining recognized keys are `label`, `cwd`, `args`, `env`, `provides`, `depends`, `outputs`, `codec`, `kind`, `priority`, and `default`. Commands are whitespace-split, list values are comma-separated, and `env` contains comma-separated `K=V` pairs. [\[crates/ostadix-api/src/project/manifest.rs, lines 25–167\]](#) [\[crates/ostadix-api/src/project/manifest.rs, lines 253–369\]](#)

```
[project]
name = "project-policy-matrix"
default_route = "a"

[[routes]]
id = "a"
command = ["/bin/echo", "42"]
provides = ["value"]
result_codec = "json"
priority = 2
default = true

[[routes]]
id = "b"
```

```

command = ["/bin/echo", "42"]
provides = ["value"]
result_codec = "json"
priority = 1

[[route_sets]]
provides = "verify-case"
alternatives = ["a", "b"]
policy = "verify_equivalent"

```

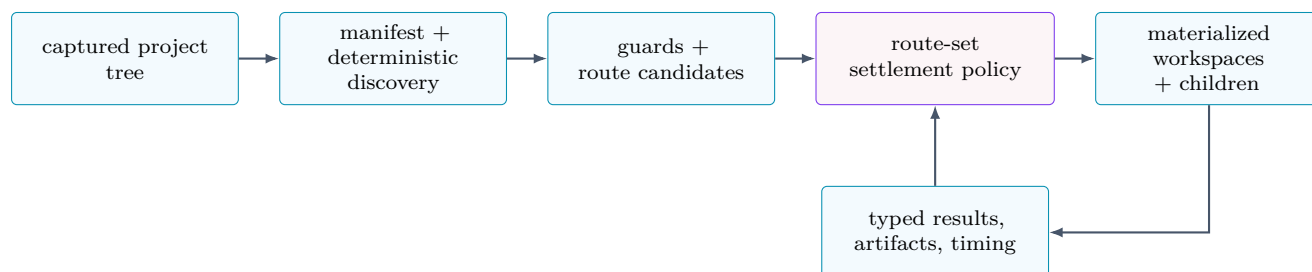


Figure 23: Project capture and policy settlement. A route set observes concrete results until its policy can settle, cancel, compare, or rank candidates.

The nine public selectors encode distinct settlement algorithms. `explicit` chooses a named route; `default` chooses the declared default; `fallback` tries routes sequentially in descending priority order; `any_success` tries declaration order; `race_success` settles on the first success; `race_settle` settles on the first completion; `all` returns every sequential result, including nonzero exits; `verify_equivalent` runs alternatives concurrently, requires success, and compares structured results; and `benchmark_and_select` retains every parallel result while placing the fastest success last as the effective selection. Route sets are declared explicitly in the project manifest. A route's `provides` metadata describes capability publication and remains independent of route-set membership.

The tracked `tests/fixtures/project_hgraph/olang.project.toml` supplies a compact authoring example. Its main route set chooses `verify_equivalent` over `impl-a` and `impl-b`; both alternatives depend on `prepare`. The compiler graph in Section 9.2 shows the resulting isolated branches, comparison operation, and final selection directly.

```

./target/release/o-link tests/fixtures/project_hgraph --project \
  -o output/current-master/o-linked-project.0
./target/release/olangc output/current-master/o-linked-project.0 \
  --target ir --route main --shim-dir backends
./target/release/olangc output/current-master/o-linked-project.0 \
  --target dot --route main --shim-dir backends

```

9.5 Unlinking

`o-unlink` restores both embedded project bundles and literal linked documents. Project restoration validates relative paths, ancestor symlinks, and in-root symlink targets before writing. Literal restoration decodes the escaped file boundaries and rejects absolute paths and parent traversal. `[src/bin/ounlink.rs, lines 49–148]`

10 Toolchain and implementation editions

10.1 Public binaries

The root Cargo package declares fourteen binaries.

Binary	Implemented role
<code>o</code>	Hosted interpreter, REPL, executor selection, compatibility backend grants, and hidden backend worker mode.
<code>o-cli</code>	Compiled lowercase intent front door for run, plan, explain, inspect, and durable run records.
<code>olangc</code>	Hosted packaging/compiler front end: binary, WASM, script, IR, and DOT.
<code>ocorec</code>	O-CORE compiler: AST, HIR, MIR, assembly, or relocatable object.
<code>o-link</code> / <code>o-unlink</code>	Literal linking and safe project bundle lift/restoration.
<code>o-notebook</code>	Feature-gated local notebook server with per-process O scope and typed result rendering.
<code>ogit</code>	Bounded semantic-receipt and policy-aware diff demonstration.
<code>o-live-host</code>	Hosted Live-World lifecycle oracle.
<code>o-node</code> / <code>octl</code>	Hosted node service, discovery, reciprocal pairing, paired TLS operation, and node-control client.
<code>o-registry</code> / <code>o-info</code>	Registry V1 event operations and Information-root inspection.
<code>ocore-kernel-world-record</code>	Verify a strict package/payload pair and encode its deterministic native admission record.

[Cargo.toml, lines 20–76]

10.2 What olangc targets mean

Target	Implementation	Execution model
<code>binary</code>	Generates a temporary Cargo project containing embedded source, runtime modules, shims, and a lockfile; builds a host executable.	The generated executable embeds the hosted runtime, reparses its source at startup, and evaluates through the ordinary graph path.
<code>wasm</code>	Uses the generated-runtime path for <code>wasm32-wasip1</code> .	Produces a WASI module containing the generated hosted runtime and embedded program.
<code>script</code>	Parses and invokes the normal evaluator inside <code>olangc</code> .	This is the direct compiler-driver entry into hosted graph evaluation.

Target	Implementation	Execution model
<code>ir</code>	Prints OIR, the execution plan, and HGRAPH.	It exposes structural lowering, frozen interfaces, dependencies, and resource state in one inspection stream.
<code>dot</code>	Solves and renders directed graph operation ports.	It visualizes executable operations and the value/resource ports connecting them.

[src/bin/olangc.rs, lines 130–258] [src/bin/olangc.rs, lines 923–1473] [src/bin/olangc.rs, lines 1910–2365]

Project inputs compile through binary, WASM, and script targets. The generated project runtime restores the captured tree and executes its selected subprocess route, carrying whole-project structure into the packaged artifact. Project directories and lifted bundles also enter the IR and DOT targets, which build the exact logical Project HGraph for the selected route or route set. A generated project WASI module contains the project runtime; its route operations use the process facilities supplied by the execution substrate.

10.3 Reference implementations

The Rust edition is authoritative and implements the full hosted graph, project, Live-World, and O-CORE surfaces. The Python edition identifies itself as the original/reference implementation at version 0.4.0. It has a compact recursive evaluator and uses environment 0 as persistent by default. The C17 edition supplies a portable hosted interpreter and hosted AOT path. Together, these editions provide independently executable semantic references around the authoritative Rust graph runtime.

[o_lang/README.md, lines 1–14] [o_lang/__init__.py, lines 31–40] [o_lang/evaluator.py, lines 1–182] [c_cpp/README.md, lines 1–133]

Edition	Parser-visible evaluator registry	Implementation surface
Rust 0.4.0 Unreleased	30 canonical tags and six aliases listed in Section 7.	OIR, plans, HGraph, both schedulers, projects, native and WASI generation, REPL, notebook, O-Git, Live-World, KernelWorld records, and O-CORE.
Python 0.4.0	<code>python</code> , <code>markdown</code> , <code>html</code> , <code>latex</code> , <code>text</code> , 0, <code>quote</code> , <code>nix</code> , <code>nix_store</code> , <code>nixos_test</code> ; aliases <code>py</code> , <code>md</code> , <code>tex</code> , <code>plain</code> , 0.	Direct execution, AST dump, text/HTML/Markdown/LaTeX/JSON projections, file output, the typed root API below, parser tests, and evaluator tests.
C17 interpreter	0, <code>python</code> , <code>html</code> , <code>latex</code> , <code>markdown</code> , <code>bash</code> , <code>shell</code> , <code>rust</code> , <code>racket</code> , <code>nix</code> , <code>nix_expr</code> , <code>nix_store</code> , <code>nixos_test</code> , <code>quote</code> .	Portable hosted parsing, evaluation, embedding headers, and backend-process execution through the C17 test corpus.
C17 AOT	The interpreter registry plus <code>text</code> .	Generated C, host compilation, hello execution, and meta-evaluation.

The Python package root exposes its complete small embedding vocabulary:

API family	Public names
Parse and evaluate	<code>run</code> , <code>parse</code> , <code>evaluate_document</code> , <code>EvalContext</code> , <code>pretty</code> .
Syntax tree	<code>Document</code> , <code>ExpressionNode</code> , <code>TextPart</code> .
Values	<code>OValue</code> , <code>ONull</code> , <code>OBool</code> , <code>OInt</code> , <code>OFloat</code> , <code>OStr</code> , <code>OHtml</code> , <code>OStorePath</code> , <code>OList</code> , <code>OMap</code> , <code>OScope</code> , <code>OBlob</code> , <code>OExpr</code> .
Conversion and rendering	<code>from_python</code> , <code>to_python</code> , <code>render_plain</code> , <code>to_json_str</code> .

[`o_lang/__init__.py`, lines 12–38]

The C17 edition publishes six headers as an embedding ABI:

Header	Public integration surface
<code>value.h</code>	Tagged <code>OValue</code> union, constructors, reference counting, maps, content identity, JSON, length-prefixed CBOR frames, SHA-256, and base64.
<code>parser.h</code>	AST storage, registered-language sets, parser lifecycle, tree destruction, and source reconstruction.
<code>eval.h</code>	Eager, lazy, and autonomous policy; evaluator lifecycle, document and source evaluation, rendering, and shim lookup.
<code>process.h</code>	Backend processes, reentrant evaluation steps, and the persistent (<i>language, environment</i>) process registry.
<code>scheduler.h</code>	Disk cache, configurable autonomous scheduler, batch callbacks, and transitive-request collection.
<code>nix_ops.h</code>	Typed instantiate, realise, activate or dry-activate, and current-system transitions.

[`c_cpp/include/value.h`, lines 13–222] [`c_cpp/include/parser.h`, lines 23–114] [`c_cpp/include/eval.h`, lines 16–54] [`c_cpp/include/process.h`, lines 13–73] [`c_cpp/include/scheduler.h`, lines 13–49] [`c_cpp/include/nix_ops.h`, lines 10–21]

10.4 REPL and notebook state

The interactive surfaces expose two deliberate state lifetimes. The REPL retains `O` lexical bindings and evaluator-owned backend actors across entries. Its `:r` command replaces the `O` lexical scope while leaving persistent actor processes available. The notebook binds `127.0.0.1:8888` and places one `Session` behind a process-wide mutex. Every browser client therefore observes the same `O` scope and evaluator registry. A POST to `/reset` replaces that complete session, clearing both lexical bindings and actor state. GET `/` serves the embedded interface, while POST `/eval` returns tagged null, text, HTML, or image output.

10.5 O-Git

O-Git demonstrates semantic receipts and policy-aware diffs. A receipt records source and target paths, group declarations, output, and qualitative properties classified as preserved, lowered, erased, or assumed. The command turns semantic transformation choices into a machine-readable artifact that can be compared alongside ordinary repository history.

[`src/bin/ogit.rs`, lines 96–183] [`src/bin/ogit.rs`, lines 523–576]

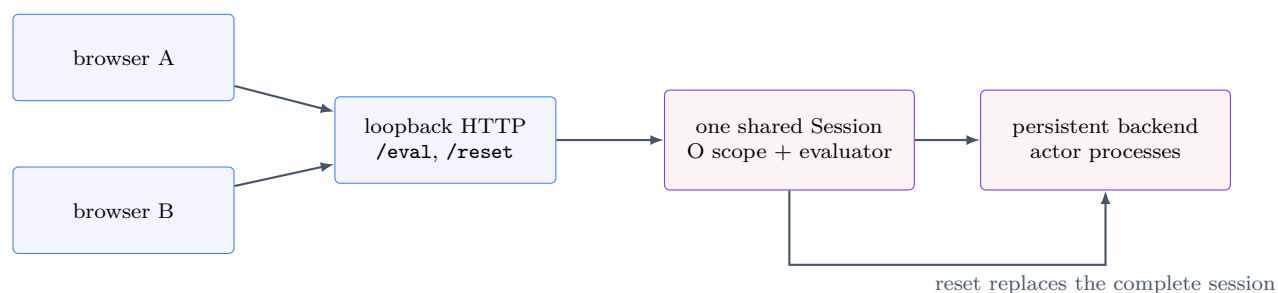


Figure 24: The local notebook has one process-wide interactive state. Cell evaluation preserves both O bindings and backend actors; reset creates a new scope and evaluator together.

The checked-in `examples/group_pipeline/main.O` and `main.eager.O` are receipt inputs that expose a lazy-to-eager policy change. O-Git lowers and executes its generated C target, then records the semantic transformation. Direct hosted O evaluation encounters their unforced `{defer}` splice, so an executable hosted variant would place `now(...)` at that force point. This is the exact semantic distinction the receipt is designed to preserve.

11 O-core: the native language

11.1 Language surface

Every `.oc` unit begins with a module declaration. The lexer and AST cover imports and visibility; functions, unsafe and extern declarations; structures, enums, statics, and constants; conditional and loop control; arrays and aggregate construction; direct calls; field and index access; casts; volatile operations; and inline assembly. [crates/ostadix-api/src/ocore/lexer.rs, lines 9–88] [crates/ostadix-api/src/ocore/parser.rs, lines 23–107] [crates/ostadix-api/src/ocore/ast.rs, lines 5–285]

Supported foreign interfaces include `extern "sysv64"`, `extern "C"`, and `extern "ocore"`. A `let` requires an initializer, assignment is right-associative, and binary precedence is encoded in the parser. Inline assembly has explicit syntax and operands. [crates/ostadix-api/src/ocore/parser.rs, lines 335–755]

The checked-in minimal unit shows the source spelling that was compiled through AST, HIR, MIR, assembly, and reproducible ELF object output in this release:

```

module examples::minimal;

@export
pub fn add(left: u64, right: u64) -> u64 {
    return left + right;
}
  
```

[ocore/examples/minimal.oc, lines 1–6]

The module declaration names the unit, `@export` fixes external visibility, `pub` admits cross-module lookup, the signature fixes every machine type, and the explicit return becomes typed SSA before x86-64 lowering. Structures, enums, imports, statics, constants, extern declarations, unsafe blocks, volatile and atomic intrinsics, and `asm!` use the same direct item-and-expression grammar across the current 132-file O-core source corpus: 77 runtime files, 46 kernel files, eight World files, and one example. [docs/OCORE.md, lines 36–75]

[docs/OCORE.md, lines 140–190]

11.2 Static semantics

Type checking proceeds in dependency-aware passes:

module scopes $\rightarrow$ type names $\rightarrow$ layouts $\rightarrow$ signatures $\rightarrow$ constants/statics $\rightarrow$ bodies.

Unsafe operations require an explicit unsafe context. Attributes are whitelisted; unknown ones are rejected. Interrupt and naked functions, aggregate calls, and floating-point ABI crossings receive explicit defensive checks. [crates/ostadix-api/src/ocore/typeck.rs, lines 16–59] [crates/ostadix-api/src/ocore/typeck.rs, lines 1881–2038]

11.2.1 Types, aggregates, and layout

The scalar set is `bool`; unsigned `u8`, `u16`, `u32`, `u64`, and `usize`; signed `i8`, `i16`, `i32`, `i64`, and `isize`; `f32`, `f64`, `void`, and `never`. Compound types are fixed arrays `[T; N]`, immutable and mutable raw pointers, declaration-ordered structures, tagged enums, and function-pointer types. Structures support construction, fields, locals, statics, and aggregate copy. Arrays support literals, repetition, indexing, locals, statics, and explicit address operations. Enum layout combines the smallest adequate `u8/u16/u32` tag with an aligned payload region.

Type class	Size (bytes)	Alignment
<code>bool</code> , <code>u8</code> , <code>i8</code>	1	1
<code>u16</code> , <code>i16</code>	2	2
<code>u32</code> , <code>i32</code> , <code>f32</code>	4	4
<code>u64</code> , <code>i64</code> , <code>usize</code> , <code>isize</code> , <code>f64</code> , pointers	8	8
<code>void</code> , <code>never</code>	0	1

Natural padding preserves declaration order. `@packed` selects alignment 1 and removes inter-field padding; `@align(N)` raises alignment to a power of two. System V scalar calls use RDI, RSI, RDX, RCX, R8, and R9, followed by stack arguments, with RAX as the scalar result and 16-byte call alignment. `@interrupt` functions return with `iretq`. [docs/OCORE.md, lines 95–193]

11.2.2 Name, conversion, storage, and call semantics

One `ocorec` invocation forms a compilation unit from one or more source files, each with a unique module name. Unqualified lookup proceeds through local bindings, current-module items, explicit imports, and predeclared intrinsics. Cross-module functions receive deterministic mangled symbols; `@export` and `@no_mangle` state the external symbol contract.

An unsuffixed integer literal inherits an expected integer type. Every other numeric or pointer conversion is written with `as`. Raw pointer arithmetic and dereference, pointer/integer conversion, mutable-static access, inline assembly, and privileged intrinsics require an explicit unsafe context. Values use deterministic destruction-free storage. Stack objects, statics, raw pointers, and explicitly linked runtime allocators state every storage lifetime at the freestanding boundary. Heap allocation occurs through an explicitly linked runtime allocator, keeping lifetime management explicit.

Direct scalar calls lower through the selected ABI, and function-pointer types carry call signatures through parsing and type checking. Aggregate parameters and aggregate returns are rejected across direct and extern call boundaries; authors pass pointers explicitly. Floating-point values have fixed storage layouts and bit-preserving transport. [docs/OCORE.md, lines 36–134] [crates/ostadix-api/src/ocore/typeck.rs, lines 2028–2038]

11.2.3 Machine operations and linkage

The volatile/atomic surface is `volatile_load`, `volatile_store`, `atomic_load`, `atomic_store`, `atomic_exchange`, `atomic_compare_exchange`, and `atomic_fetch_add`. Orders are `relaxed`, `acquire`, `release`, `acq_rel`, and `seq_cst`; the type checker validates operation width, mutability, and legal order combinations before code generation. Hardware intrinsics cover byte/word/dword port input and output, interrupt enable/disable, halt, page invalidation, `syscall10` through `syscall16`, and Intel-syntax `asm!` with explicit register operands and memory/stack options.

The linkage attributes are `@export`, `@no_mangle`, `@link_section("name")`, `@align(N)`, `@used`, `@packed`, `@interrupt`, and `@naked`. Together they let O-core source state symbol visibility, exact symbol names, ELF placement, retention, layout, and entry/exit discipline directly. `@unsafe_linkage` explicitly admits a writable static placed in an executable section. [docs/OCORE.md, lines 170–192]

11.3 Compiler pipeline

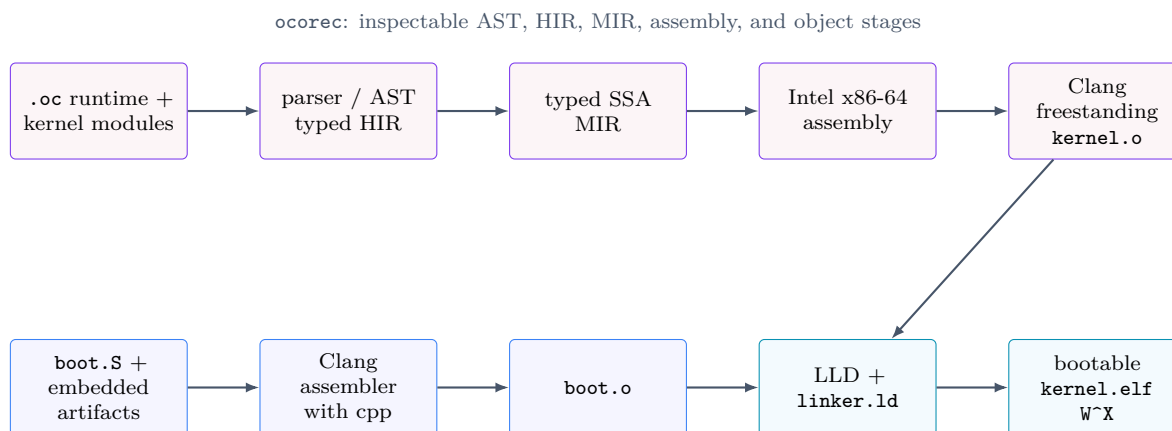


Figure 25: The native build has two feeders. `ocorec` compiles O-core modules into a freestanding object; boot assembly, embedded mode artifacts, and the linker script complete the Multiboot2/Xen-PVH kernel image.

MIR is typed SSA, while mutable locals and statics remain explicit memory places. The code generator emits Intel-syntax x86-64 and gives every local and SSA value stack storage. Clang assembles with `x86_64-unknown-none-elf`, freestanding settings, and the stack protector disabled. [crates/ostadix-api/src/ocore/mir.rs, lines 1–173] [crates/ostadix-api/src/ocore/codegen.rs, lines 1–20] [crates/ostadix-api/src/ocore/codegen.rs, lines 162–317] [crates/ostadix-api/src/ocore/driver.rs, lines 126–183]

11.4 Current compiler profile

The current backend targets freestanding x86-64 under the System V AMD64 ABI. Its lowering uses stack storage for locals and SSA values, direct calls, explicit aggregate layouts, integer and pointer operations, volatile memory, and inline assembly. Programs manage memory through statics, stack objects, pointers, and explicitly linked runtime facilities. [docs/OCORE.md, lines 351–365]

Native compiler substance

O-CORE has a complete parser, layout-aware type checker, typed SSA MIR, defensive ABI rules, deterministic x86-64 assembly, and reproducible ELF object tests. That pipeline makes the language executable at the machine boundary and provides the source language for the Ostadix freestanding runtime and kernel.

12 Kernel, capabilities, and native execution

12.1 Freestanding kernel substrate

The kernel build links a complete set of `.oc` runtime modules with boot assembly and a linker script. It uses Rust/Cargo `ocorec` on the host, Clang for assembly, and LLD for ELF linking. The image contains Multiboot2 and Xen PVH metadata, `W^X` program headers, fixed user RX and RW-NX regions, guard-page checks, and fail-closed initialization of memory, process/domain, scheduler, capability, mapping, and IPC state. [\[ocore/kernel/build.sh, lines 358–529\]](#) [\[ocore/kernel/linker.ld, lines 1–105\]](#) [\[ocore/kernel/main.oc, lines 180–496\]](#)

12.2 Native capability model

Native handles encode

$(generation \ll 32) \mid slot,$

while CSpace identity remains kernel-private. Validation checks the current CSpace, slot, occupancy, generation, object kind, and rights. Reservation and close are explicit transactions; generation exhaustion retires a slot. Memory-view and delegated-resource authority is deliberately nontransferable. [\[ocore/runtime/x86_64/capability.oc, lines 21–69\]](#) [\[ocore/runtime/x86_64/capability.oc, lines 294–394\]](#) [\[ocore/runtime/x86_64/capability.oc, lines 520–543\]](#) [\[ocore/runtime/x86_64/capability.oc, lines 752–880\]](#)

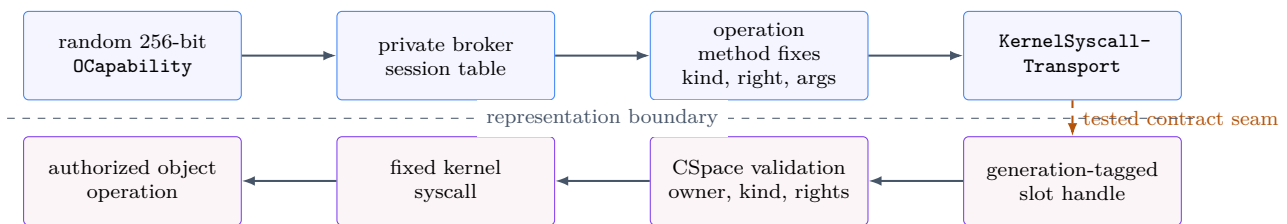
The kernel syscall contract fixes authority checks to seventeen numbered operations. A caller supplies generated handles; the dispatcher derives the current PCB and CSpace, validates object kind, generation, and rights, and then performs the operation.

No.	Operation	Contract
0	<code>debug_write</code>	Capability-gated bounded copy from readable user memory.
1	<code>cap_close</code>	Close one generation-tagged CSpace entry.
2	<code>cap_copy</code>	Prepare an attenuation-only transfer ticket to an authorized endpoint's receiver.
3	<code>page_alloc</code>	Allocate a typed page object and return a kernel-selected capability.
4	<code>yield</code>	Record or perform a cooperative scheduler transition.
5	<code>ticks</code>	Return the diagnostic timer count.
6	<code>exit</code>	Enter the configured trusted lifecycle continuation.
7	<code>sleep</code>	Block until a scheduler-managed tick deadline.
8	<code>endpoint_create</code>	Create and return a generation-safe endpoint.
9	<code>endpoint_send</code>	Send two words plus an optional transfer ticket.
10	<code>endpoint_receive</code>	Receive one bounded 32-byte message.

No.	Operation	Contract
11	<code>endpoint_cancel</code>	Cancel by endpoint and correlation identity.
12	<code>serial_read</code>	Read one nonblocking byte through the exact control capability.
13	<code>control_submit</code>	Submit one checked command of at most 192 bytes.
14	<code>personality_call</code>	Invoke the M6A scalar route with a deadline.
15	<code>personality_reply</code>	Complete an exact M6A request through a reply capability.
16	<code>personality_supervise</code>	Apply an M6A policy action to an exact generation and subject.

The message path uses fault-aware bounded copy, FIFO endpoint queues, real TCB block/wake epochs, request correlation, lifecycle cancellation, and exact attenuation. The scalar personality calls use distinct call, reply, and supervision capability kinds, so operation role is carried by authority rather than a caller-provided mode bit. [docs/OCORE.md, lines 205–284] [docs/OCORE.md, lines 442–496]

hosted identity



native authority

Figure 26: The bridge preserves hosted bearer identity and kernel-owned native authority. Operation-specific contracts cross the transport seam; CSpace validation resolves generation, object kind, and rights before execution. A private session table owns bearer-to-handle bindings, while recording and failure transports verify the exact calls.

[crates/ostadix-api/src/ocore/capability_bridge.rs, lines 1–159]

13 Live-World and KernelWorld

hosted Live-World

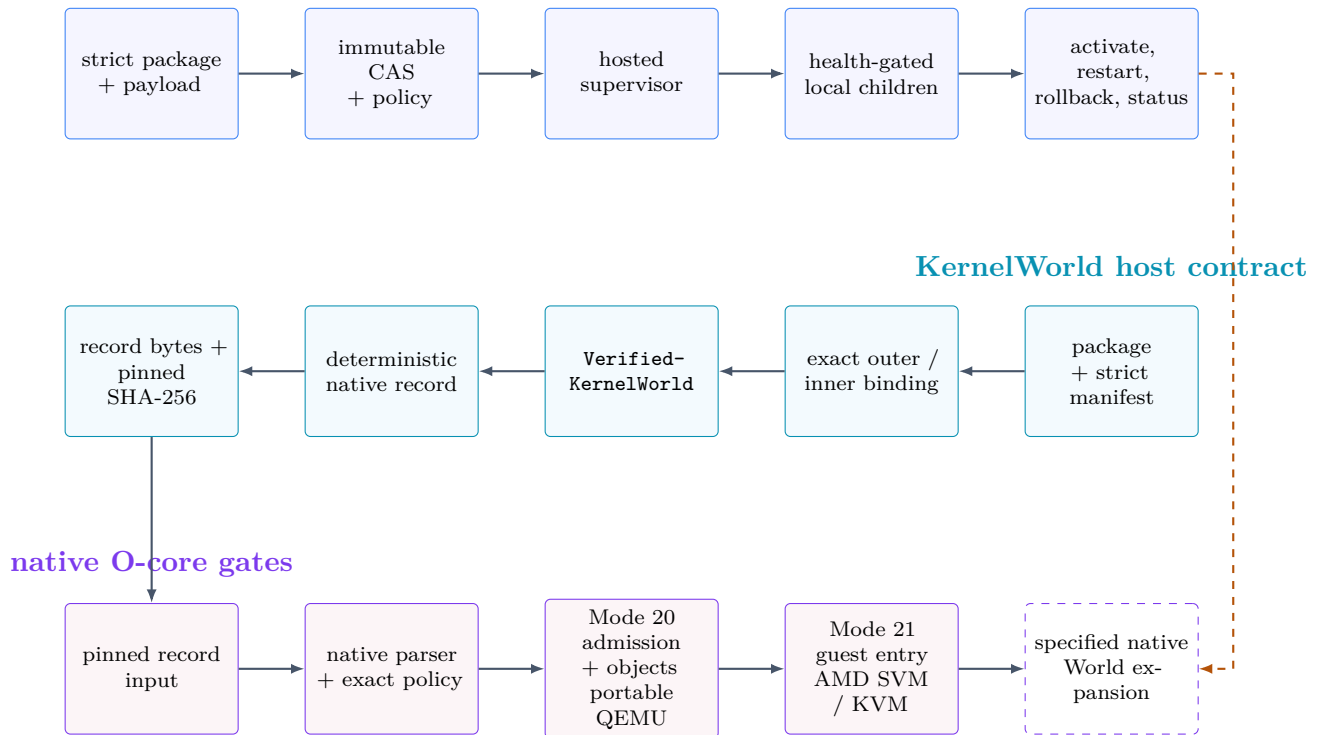


Figure 27: World semantics, host verification, and native execution occupy separate substrates. The record path is a solid authority chain into native admission. The dashed path marks differential comparison between hosted lifecycle behavior and the specified native continuation.

13.1 Three different layers

Layer	Implemented substrate	Architectural role
Hosted Live-World	Local child processes, immutable content store, strict manifests, policy, health checks, rollback/restart/reconstruction, and private bearers.	Executable lifecycle semantics and a differential oracle for native World stages.
Native M5 slice	Separately loaded x86-64 ELF services and CSspaces under QEMU, a serial control loop, and generation-based crash/restart behavior.	Machine-level service isolation, capability routing, crash recovery, and generation replacement in the M5 execution gate.
Target architecture	Independent unprivileged endpoint-RPC services, durable reconstruction, and a native build service.	The specified expansion path from the implemented native service substrate to a complete World lifecycle.

Hosted advisory locks and revision compare-and-swap serialize lifecycle transactions over the local immutable content store. Supervisor code implements generation rotation, bearer issuance, exact live-binding invocation, restart, and stale-generation denial. [crates/ostadix-api/src/live_system/supervisor.rs, lines 770–1007]

13.2 Live-World command and state model

The public `o-live-host` surface is a complete local lifecycle. A package enters an immutable content-addressed store, activation creates a health-checked service generation, invocation resolves the exact active binding, composition passes OVALUE between worlds, and every mutation records enough state for reconstruction.

Command	State transition	Resulting state
<code>pack</code>	Canonical manifest and payload → exact package digest.	Emits the content identity calculated over canonical manifest semantics and the exact captured payload tree.
<code>install</code>	Verified package → immutable CAS object.	Writes the object and strict alias metadata into the content-addressed store.
<code>activate</code>	Policy admission + health → active generation.	Publishes a health-checked service generation and its exact policy grants.
<code>invoke</code>	Active bearer + ABI + operation → tagged result.	Resolves the exact live binding and returns a bounded tagged OVALUE.
<code>compose</code>	Invoke chain → OVALUE transfer between worlds.	Passes each typed result into the next declared service operation.
<code>upgrade</code>	Candidate health + generation rotation → new active digest.	Retains the prior healthy root and publishes fresh service generations.
<code>rollback</code>	One retained prior healthy root ↔ current package, with fresh service generations.	Swaps the retained and active roots after health and policy validation.
<code>restart</code>	Selected service → fresh process and bearer generation.	Rotates the service generation and invalidates the prior live binding.
<code>status</code>	Revisioned <code>active-set.json</code> + CAS + policy → reconstructed and health-checked active set.	Reconstructs exact package digests, services, generations, and policy grants from those three state sources.
<code>demo</code>	Fixed acceptance scenario → nine lifecycle markers.	Exercises immutable CAS, policy denial, health, composition, rollback, stale-bearer rejection, crash isolation, restart, and reconstruction.

13.3 Authorable Live-World schemas

The lifecycle is driven by four strict public data surfaces. These are the formats a package author and operator write directly.

Surface	Required shape
Package manifest, TOML	<code>schema = "ocore.package/v1"</code> plus required <code>name</code> , <code>version</code> , <code>architecture</code> , <code>payload_sha256</code> , <code>runtime</code> , <code>services</code> , <code>capability_requests</code> , <code>health</code> , and <code>build</code> . Runtime contains <code>kind</code> , <code>entry</code> , and <code>abi</code> ; services contain <code>name</code> and <code>protocol</code> ; requests contain <code>kind</code> , <code>rights</code> , and <code>purpose</code> ; health contains <code>protocol</code> and <code>timeout_ms</code> ; build contains <code>source_sha256</code> and <code>builder</code> .
Activation policy, TOML	<code>schema = "ocore.hosted-activation-policy/v1"</code> plus <code>grants</code> . Admission selects the exact (<i>package</i> , <i>kind</i> , <i>purpose</i>) grant and requires its rights set to contain every requested right.
Runtime program, TOML	Required <code>schema = "ocore.runtime-program/v1"</code> , <code>world</code> , <code>health.status</code> , and at least one operation, where status is <code>healthy</code> or <code>unhealthy</code> . The kinds are fieldless <code>identity</code> and <code>crash</code> , <code>wrap.field</code> , <code>prefix.text</code> , <code>int_pair.lhs/rhs</code> , <code>sum_fields.lhs/rhs</code> , and <code>fail.message</code> .
Composition plan and input, JSON	A nonempty array of { <code>service</code> , <code>protocol</code> , <code>operation</code> } steps receives one tagged OVALUE JSON input. Each result becomes the next step's input.

Package verification establishes the general `ocore.package/v1` contract. Hosted activation and reconstruction then require `architecture` to equal the host architecture, `runtime.kind` to equal `native_test_runtime`, `runtime.abi` to equal `ocore.runtime-service/v1`, and `health.protocol` to equal `ocore.health/v1`. The runtime entry is an absolute path inside the captured payload, such as `/bin/live`. Manifest health timeouts range from 1 through 60,000 ms; the public CLI's default supervisor admits at most 5,000 ms.

```
schema = "ocore.runtime-program/v1"
world = "example.source"
```

```
[health]
status = "healthy"
```

```
[operations.produce]
kind = "int_pair"
lhs = 20
rhs = 22
```

```
schema = "ocore.hosted-activation-policy/v1"
```

```
[[grants]]
package = "runtime/example"
kind = "endpoint"
purpose = "request channel"
rights = ["send", "receive"]
```

```
[
  {"service": "world.source", "protocol": "ocore.runtime-service/v1",
    "operation": "produce"},
  {"service": "world.sum", "protocol": "ocore.runtime-service/v1",
    "operation": "compute"}
]
```

The plan's first input can be `{"t": "null"}` or any other tagged OVALUE JSON value. The worker protocol carries the same algebra in bounded canonical-CBOR frames, preserving type and structure between processes. Unknown manifest, policy, runtime-program, and composition fields fail strict deserialization before lifecycle mutation. [crates/ostadix-api/src/live_system/manifest.rs, lines 35–90] [src/bin/o-live-host.rs, lines

35–39] [src/bin/o-live-host.rs, lines 149–163] [src/bin/o-live-host.rs, lines 381–411] [crates/ostadix-api/src/live_system/protocol.rs, lines 20–73] [crates/ostadix-api/src/live_system/supervisor.rs, lines 168–188] [crates/ostadix-api/src/live_system/manifest.rs, lines 554–577] [crates/ostadix-api/src/live_system/supervisor.rs, lines 1076–1092] [crates/ostadix-api/src/live_system/supervisor.rs, lines 1469–1492]

The runtime owns three further strict record families that connect installed bytes to reconstructed services and typed process exchange.

Runtime-owned format	Exact shape
<code>ocore.store-object/v1</code> and <code>ocore.store-alias/v1</code> , TOML	An object records <code>schema</code> , <code>algorithm</code> , and <code>digest</code> . An alias adds alias; both select <code>sha256</code> .
<code>ocore.hosted-active-set/v1</code> , JSON	<code>schema</code> , <code>monotonic revision</code> , <code>active</code> , and <code>rollback</code> . Each package record carries <code>package_name</code> , <code>digest</code> , and <code>services</code> ; each service carries <code>name</code> , <code>protocol</code> , and <code>generation</code> . One prior healthy root can be retained per active package.
<code>ocore.runtime-service/v1</code> , canonical CBOR	Requests tagged by <code>type</code> : <code>health{nonce}</code> , <code>invoke{service, operation, input}</code> , and <code>shutdown</code> . Responses are <code>healthy{nonce, world}</code> , <code>unhealthy{nonce, message}</code> , <code>result{value}</code> , <code>error{message}</code> , and <code>stopped</code> .

[crates/ostadix-api/src/live_system/store.rs, lines 16–23] [crates/ostadix-api/src/live_system/store.rs, lines 109–124] [crates/ostadix-api/src/live_system/supervisor.rs, lines 201–238] [crates/ostadix-api/src/live_system/protocol.rs, lines 75–102]

Exact package identity is constructed from canonical semantics and captured bytes. The manifest first becomes canonical identity bytes. Payload entries are normalized and sorted, then encoded with a length-prefixed relative path, Unix executable bit, and file contents. Separate `ocore.payload-tree/v1` and `ocore.package-object/v1` SHA-256 domains distinguish the tree hash from the complete package hash. On Unix, payload traversal is descriptor-relative and applies `O_NOFOLLOW` at every component. The scanner retains opened regular-file descriptors through capture and compares metadata around the read. The resulting digest therefore names the canonical manifest and the exact files opened for capture, and that identity becomes the immutable CAS key used by activation, rollback, and reconstruction.

[crates/ostadix-api/src/live_system/manifest.rs, lines 602–930] [crates/ostadix-api/src/live_system/manifest.rs, lines 1454–1482]

Layer	Implemented fixed capacities
Hosted Live-World	64 KiB manifest; 128-byte logical name; 64-byte identifiers; 256-byte protocol; 512-byte purpose; 4,096-byte runtime entry; 32 services; 64 capability requests with 16 rights each; 8,192 payload entries including 4,096 files; 32 path components of 255 bytes; 64 MiB per file; 256 MiB per package; 4 KiB store metadata.
Hosted worker protocol	1 MiB runtime program and frame; 256 operations; 4,096-byte operation messages.

Layer	Implemented fixed capacities
Hosted supervisor	The public CLI defaults to a 5-second health ceiling, 5-second invocation timeout, and 64 KiB structural-value limit. Configurable maxima are 60 seconds and 512 KiB; public input files have a separate 512 KiB read ceiling. Fixed bounds are a 1 MiB active-set file, 64 active packages, 256 active services, 64 rollback roots, 4,096 live bearers, 256 composition steps, 255-byte operation names, and a 64 KiB activation policy with 256 exact grants.
KernelWorld host contract	64 KiB manifest; 64 exports; 64 requests; 32 requirements; 16 rights per request; 128-byte identifiers; 512-byte text fields; 256 vCPUs; 1,048,576 MiB declared memory; 65,536 outstanding requests; 1,000,000 requests per generation; 1 TiB shared memory; 256 devices; health timeout from 1 through 60,000 ms; 16 KiB native record.
Native admission	The native record admits four exports and eight requests. Runtime admission holds two worlds, four exports and eight requests per world, 16 policy rules, and 512-byte policy fields. Mode 20 seals two VM, four vCPU, and eight guest-page objects in its executed gate.

[crates/ostadix-api/src/live_system/supervisor.rs, lines 40–163] [src/bin/o-live-host.rs, lines 35–39]

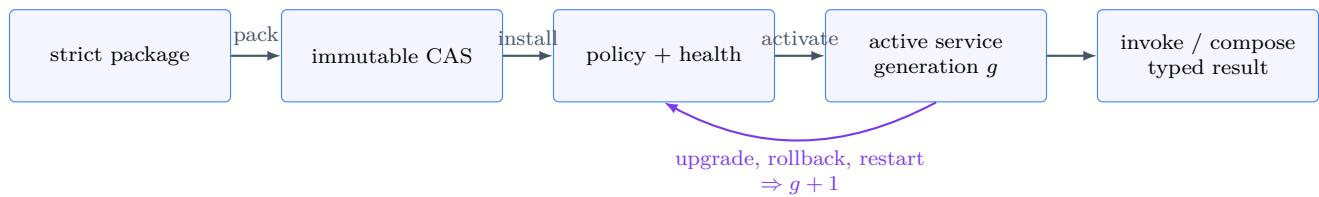


Figure 28: Admission and health create an executable Live-World generation. Upgrade, rollback, and restart return through the gate and allocate a new generation, making stale bearer identity explicit.

13.4 KernelWorld authority chain

Host verification joins the outer `ocore.package/v1` object to the inner world manifest through exact equality. The outer runtime declares `kind = "kernel-world"`; its ABI is `ocore.kernel-world-control/v1`. Its absolute payload entry names a non-executable UTF-8 manifest file. Outer and inner name, version, architecture, health protocol, health timeout, service name/protocol set, and normalized capability-request set match exactly. For a `package_payload` image, the declared SHA-256 equals the digest of the exact image bytes opened from the verified payload. These equalities produce one `VerifiedKernelWorld` bound to the immutable package digest. That verified object creates the authority-bearing native record, whose native codec checks every encoded bound exactly and rejects overflow. [crates/ostadix-api/src/kernel_world.rs, lines 1–7] [crates/ostadix-api/src/kernel_world.rs, lines 248–275] [crates/ostadix-api/src/kernel_world.rs, lines 611–749]

The authorable KernelWorld manifest is strict TOML under `ocore.kernel-world/v1`. Its required top-level fields are `schema`, `name`, `version`, `integration`, `image`, `machine`, `lifecycle`, `quotas`, `exports`, `capability_requests`, and `license`.

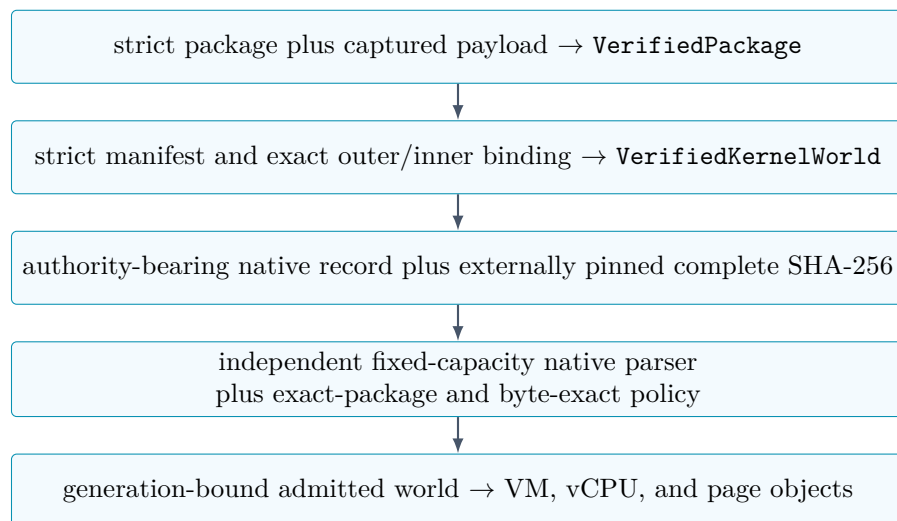


Figure 29: KernelWorld verification and native admission.

Field group	Public vocabulary and cross-field contract
Integration and image	<code>source_integrated</code> uses a <code>package_payload</code> image, <code>paravirtual</code> execution, and <code>direct</code> firmware. <code>binary_contained</code> uses <code>hardware_virtualized</code> execution and requests <code>vm.machine</code> run authority. Image kinds are <code>package_payload</code> with <code>path/sha256</code> , or <code>user_supplied</code> with <code>expected_sha256</code> .
Machine	<code>guest_architecture</code> , <code>profile</code> , <code>execution</code> <code>paravirtual hardware_virtualized</code> , firmware <code>direct uefi</code> , bounded min/max vCPUs and memory, and requirements.
Lifecycle and quotas	<code>health_protocol</code> , <code>health_timeout_ms</code> , <code>restart</code> <code>never on_failure always</code> ; outstanding requests, requests per generation, shared-memory bytes, and device ceilings. <code>max_requests_per_generation</code> is at least <code>max_outstanding_requests</code> .
Exports and authority	Both lists are nonempty. Each export has <code>name</code> , <code>plane</code> <code>device abi semantic</code> , and <code>protocol</code> . Each capability request has <code>kind</code> , one or more <code>rights</code> , and <code>purpose</code> . A device-plane export requires a <code>device.*</code> request and a nonzero device quota.
License	Redistribution is <code>redistributable user_supplied_only</code> , paired with <code>external_acceptance_required</code> . A user-supplied image selects <code>user_supplied_only</code> and requires external acceptance.

[crates/ostadix-api/src/kernel_world.rs, lines 18–182] [crates/ostadix-api/src/kernel_world.rs, lines 277–444] [crates/ostadix-api/src/kernel_world.rs, lines 408–568]

The same public host contract includes `KernelWorldInstance`, a bounded semantic oracle for lifecycle and request settlement. Its state path is `Installed` → `Starting` → `Healthy`, with explicit `Failed` and `Stopped` transitions. The manifest’s `never`, `on_failure`, or `always` restart policy decides which terminal state may begin a fresh generation. `Healthy` export resolution returns immutable package, world, integration-mode, generation, and export provenance.

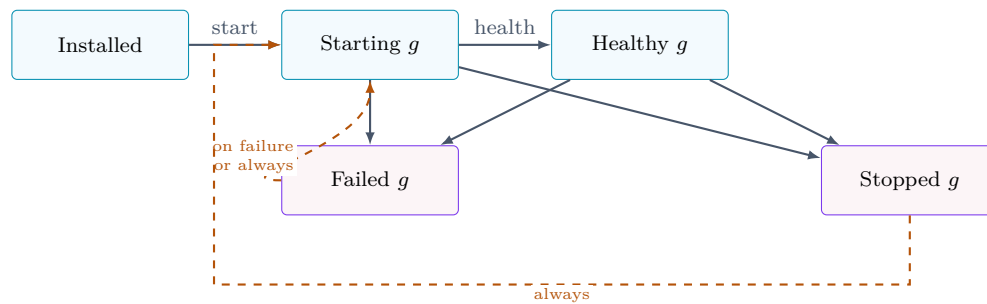


Figure 30: The host-side KernelWorld lifecycle oracle. Restart policy controls generation replacement, while health controls export publication.

Every `RequestId` is the pair $(generation, sequence)$. Outstanding and per-generation quotas gate admission before insertion. Reply, cancellation, deadline expiry, world failure, and world stop settle requests exactly once as `Replied`, `Cancelled`, `DeadlineExceeded`, `WorldFailed`, or `WorldStopped`. Failure and stop drain all in-flight requests into the corresponding terminal record; a replacement generation resets the sequence and makes every earlier request ID stale. [crates/ostadix-api/src/kernel_world.rs, lines 1208–1570]

Native admission consumes the deterministic normal form under an independently registered immutable policy. It validates exact package identity and byte content by kind and purpose, then creates a generation-bound grant after every rule and cross-field constraint succeeds. The grant remains `ADMITTED` until exact-generation revocation. [ocore/runtime/x86_64/kernel_world_admission.oc, lines 1–11] [ocore/runtime/x86_64/kernel_world_admission.oc, lines 1375–1478] [ocore/runtime/x86_64/kernel_world_admission.oc, lines 1813–1934]

13.5 Native execution ladder

The native gates form one dependency-ordered execution ladder. Modes 0–20 establish the x86-64 bootstrap, isolation, process, scheduler, IPC, service, personality, and KernelWorld prerequisites consumed by the later modes. Each row names the new machine fact introduced at that point.

Mode	Mode contract
0	Long-mode bootstrap, O-core kernel entry, serial output, allocator, IDT, PIC/PIT, syscall entry, and one linked CPL3 personality.
1–8	Fatal isolation probes for divide, invalid opcode, non-present access, supervisor-page read, guard-stack access, NX instruction fetch, noncanonical address, and invalid syscall return.
9	Recoverable guarded <code>copy_from_user/copy_to_user</code> fault path.
10	Normal process exit, exact lifecycle teardown, stale-handle denial, sibling survival, and reclamation.
11	Contained process-fault teardown with the unrelated sibling continuing.
12	Four-TCB scheduler across two processes, preemption, yield, sleep/wake, lifecycle, hostile saved-state containment, and one million identity switches.
13	Kernel-side endpoint and capability-transfer foundation.
14	Public CPL3 IPC with four-message FIFO blocking/wake, correlation, attenuated transfer, death cleanup, and personality containment.
15	Deterministic OVFS, static ELF loading, BSS/stack construction, W^X mappings, and service namespace.

Mode	Mode contract
16	Four loaded service ELF's, serial activation, contained package-daemon fault, health-gated restart, and final reclamation.
17	Immutable package semantics, exact rollback, stale-generation denial, restart, and control-parser behavior.
18	Four-principal scalar personality RPC, unprivileged supervisor, timeout/death settlement, restart, and exact capability rebind.
19	Four request-scoped bounded-copy views, five lease classes, written-prefix commit, terminal ordering, and delegated revocation.
20	Exact KernelWorld record parsing, default-deny admission, and generation-bound VM, vCPU, and guest-page object construction.
21	AMD SVM/NPT synthetic-vCPU entry, controlled exit, injection, denied GPA, teardown, and restart through the dedicated hardware gate.

The implemented capacities make each native proof repeatable: 3,072 reclaiming frames cover the fixed 4–16 MiB pool; M2 uses four TCBs, two processes, and one CPU; M3 exposes four FIFO messages and 16 transfer tickets; M6A uses four principals with 16 tombstones and nine retained identities; M6B provides four views of 128 bytes each under a 256-byte aggregate charge and five lease classes; Mode 20 constructs two VMs, four vCPUs, and eight guest pages.

The native artifact builders domain-separate OVFS, package, manifest, record, and image identities. Each build gate recomputes those identities from the exact current inputs and passes them to an independent validator before QEMU consumes the artifact. Reproducers should retain the hashes printed by their current-master run beside the gate transcript. Each source coordinate produces its own artifact identities.

[ocore/kernel/build-m4-artifacts.sh, lines 1–40] [ocore/kernel/build-m5-artifacts.sh, lines 1–40] [ocore/kernel/build-m6-artifacts.sh, lines 1–40]

13.6 Mode 20 and Mode 21

Gate	Implemented claim	Substrate	Execution significance
Mode 20	Exact record validation, default-deny admission, generation-bound nonexecuting VM/vCPU/pages, quota/overlap/stale/reclaim behavior.	Portable x86 QEMU gate.	Establishes the admission and object-construction stage before guest entry.
Mode 21	Real AMD SVM/NPT entry of a two-page synthetic real-mode guest, injected vector, known computation, VMMCALL, denied unmapped GPA, teardown and restart.	Writable /dev/kvm, AMD SVM/NPT, QEMU KVM -cpu host.	Defines the hardware execution gate for admitted objects and its required execution substrate.

[ocore/kernel/smoke-kernel-world-qemu.sh, lines 128–186] [ocore/kernel/smoke-kernel-world-execution-qemu.sh, lines 8–177]
[ocore/runtime/x86_64/svm_execution.oc, lines 24–507]

13.7 Modes 22 through 34

Modes 22 through 34 extend the portable evidence ladder through supervised KernelWorld execution, bounded foreign-personality services, canonical World records, causal fallback, hosted-to-native receipt convergence, challenged boot information, and four-CPU startup. All thirteen modes are required gates in the current evidence registry.

Mode	Implemented mechanism	Required gate
22	Health-gated typed export publication, reset intent, failure withdrawal, exact VM-graph revocation, generation-2 restart, stale-generation denial, and unrelated-service survival.	<code>smoke-kernel-world-live-qemu.sh</code>
23	TCG-emulated SVM/NPT entry, VMEXIT-derived health, validated virtual PIO, NPF quiescence, execution-pin release, generation rebind, and cross-World authority rejection.	<code>smoke-kernel-world-execution-device-qemu.sh</code>
24	Four digest-pinned CPL3 bounded-personality services, terminal lifecycle matrix, contained daemon fault, generation replacement, and request-wide authority cleanup.	<code>smoke-live-bounded-personality-qemu.sh</code>
25	Exact static Linux x86-64 ELF execution in CPL3 through a four-call personality, stdout/stderr results, exact <code>-ENOSYS</code> , exit status, replacement, and resource reclamation.	<code>smoke-live-linux-personality-qemu.sh</code>
26	Native Linux 9P2000 provider and Plan-9-style client over the exact <code>/srv/linux/status</code> exchange, namespace withdrawal, provider replacement, stale denial, and reclamation.	<code>smoke-live-linux-plan9-qemu.sh</code>
27	Shared Rust and O-core World identity types, byte-identical OWIDENT V1 corpus, nonzero coordinate validation, and hierarchical stale and logical comparison.	<code>smoke-world-identity-qemu.sh</code>
28	OWPROTO V1 big-endian records, exact bounds and fields, deterministic version negotiation, and Rust/O-core byte convergence.	<code>smoke-world-protocol-qemu.sh</code>
29	OWVALUE V1 portable allowlist, bounded recursive shape, canonical ordering, authority-bearing value rejection, stable hashing, and cross-language byte convergence.	<code>smoke-world-value-qemu.sh</code>
30	OWRECEIPT V1 canonical receipt and signing preimage, hosted Ed25519 sign/verify, structural signature-envelope validation, World generation and commit fields, and Rust/O-core byte convergence.	<code>smoke-world-receipt-qemu.sh</code>

Mode	Implemented mechanism	Required gate
31	One LogicalRead across generation-distinct providers, terminal A failure, A-route withdrawal, fresh B-local session reconstruction, immutable object hash validation, causal attempt state, cleanup, and witness survival.	<code>smoke-m7b-logical-read-qemu.sh</code>
32	Hosted project execution through a caller-supplied World view, live signed uncommitted receipt emission, native full decode and exact re-encode, signing preimage reconstruction, and signer-independent semantic digest equality.	<code>smoke-world-project-runtime-qemu.sh</code>
33	Challenged UEFI/Multiboot2 handoff into bounded kernel-owned BootInfo, memory-map-selected allocator window, retired inspection aperture, CPL3 lifecycle, and transcript challenge binding.	<code>smoke-x86_64-boot-info-qemu.sh</code>
34	Exact four-CPU ACPI/MADT topology, PIT progress, x2APIC INIT/SIPI, distinct AP stacks, trampoline W ^X lifecycle, one atomic BSP/AP barrier, post-barrier timer, and one-CPU rejection control.	<code>smoke-x86_64-smp-qemu.sh</code>

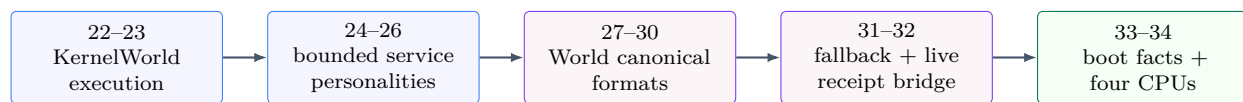


Figure 31: Modes 22 through 34 move from supervised execution through portable World records and causal recovery into challenged platform initialization.

The evidence registry declares its schema, required count, supplemental count, portable aggregate command, per-gate tool requirements, positive claims, and expected transcript markers. The validator checks registry structure, required membership, script presence, evidence classes, milestone uniqueness, marker discipline, and the release projection consumed by the portable aggregate. Mode 32 additionally wires a focused hosted project execution test into the native receipt gate so the receipt being checked originates from an executed Project HGraph path.

[evidence/gates.toml, lines 1–4] [evidence/gates.toml, lines 674–684] [evidence/gates.toml, lines 697–707] [evidence/gates.toml, lines 468–478] [evidence/gates.toml, lines 497–507] [evidence/gates.toml, lines 531–541] [evidence/gates.toml, lines 103–113] [evidence/gates.toml, lines 125–135] [evidence/gates.toml, lines 148–158] [evidence/gates.toml, lines 174–184] [evidence/gates.toml, lines 579–589] [evidence/gates.toml, lines 205–215] [evidence/gates.toml, lines 29–40] [evidence/gates.toml, lines 52–63] [scripts/release_evidence.py, lines 20–29] [scripts/release_evidence.py, lines 118–291] [scripts/release_evidence.py, lines 487–586] [ocore/kernel/smoke-world-project-runtime-qemu.sh, lines 16–35] [tests/project_world_runtime.rs, lines 359–449]

14 Reproducibility and current-master evidence

This paper pins source identity before interpreting output. The current qualification coordinate is commit 36787b16476bc0c8c4dddf665c7228b314d04e716, package 0.4.0 Unreleased. Every result below belongs to that coordinate or to an artifact derived from it.

14.1 Evidence classes

A reader should distinguish four kinds of evidence used in this paper:

1. **Defining source.** Narrow file and line citations identify the implementation that makes a claim true.
2. **Generated structure.** Compiler output exposes the exact program graph in the compiler's own vocabulary.
3. **Registry validation.** The evidence validator checks the declared gate inventory and the portable release projection.
4. **Execution transcripts.** Each registered gate defines required tools and expected markers for its actual execution substrate.

This order matters. Source establishes the mechanism. Generated structure shows the mechanism instantiated for a program. Registry validation establishes the release's required proof surface. Execution transcripts establish that a particular substrate completed the registered gate.

14.2 Current snapshot checks

Check	Observed current result	Meaning
<code>git rev-parse HEAD</code>	36787b16476bc0c8c4ddf665c7228b314d04e716	Pins every source citation and generated artifact in this paper.
Root package coordinate	<code>o-lang 0.4.0</code> ; changelog state Unreleased .	Names the synchronized development package and its release state.
Workspace ownership	Two default members; exact <code>ostadix-api =0.4.0</code> dependency; separately locked MCP root excluded.	Confirms the engine, CLI shell, and MCP dependency boundaries.
Public binary boundary	All fourteen binaries executed: thirteen printed their public help; the notebook served its loopback UI, evaluated <code>text^(42)_text</code> as typed text "42", and completed reset.	Exercises every declared root binary through a safe public entry boundary.
Root Rust quality gates	Formatting, check, and Clippy passed.	Qualifies the synchronized root and engine workspace under the current feature surface.
Release binary surface	All workspace binaries built with all features and the locked dependency graph in 3 minutes 13 seconds.	Qualifies the optimized deployable command surface independently from test harness compilation.

Check	Observed current result	Meaning
Rust test inventory	1,351 passed, 3 ignored across 62 harnesses; 1,354 tests listed.	Exercises all targets and features across hosted semantics, evidence, projects, networking, Information, Registry, World, Live-World, KernelWorld, and O-CORE host contracts.
MCP Rust package	Build and Clippy passed; 29 Rust tests passed; ten-tool smoke passed.	Qualifies the separately locked MCP package and every declared tool route.
Python reference	18 of 18 parser tests and 46 of 46 evaluator tests passed; compileall passed.	Exercises the reference grammar, evaluator, and importable source tree.
C17 edition	Make manifest: 5 passed; AOT manifest: 3 passed, 2 manifest skips; safety gate passed; CMake/CTest: 4 of 4 passed.	Qualifies the standalone interpreter, generated-C path, and both build-system surfaces.
Examples	39 Rust example runs passed, 5 manifest skips; recursive manifest accounted for 44 examples.	Exercises the tracked hosted teaching corpus under its declared platform requirements.
Source release	77 of 77 source-release checks passed.	Qualifies the current tracked source projection, package boundaries, and archive rules.
olangc target matrix	All five targets completed their compilation or emission contract; binary and script targets also executed.	Exercises host binary generation and execution, WASI generation, direct script execution, IR inspection, and DOT graph emission.
Project, Live, and World smokes	Project HGraph, project execution, Hosted Live-World, and World smoke families passed.	Connects logical planning to execution, lifecycle supervision, World codecs, and receipt paths.
Docker probe	Passed.	Qualifies the maintained container build and hosted probe boundary.
MCP tool inventory	Ten <code>#[tool]</code> definitions.	Confirms the complete agent-facing operation vocabulary listed in Section 3.4.

Check	Observed current result	Meaning
Project lift	7,501 bytes; SHA-256 a6d97fd5b95b829c3e3a754722767960f2963e0644ba8c89da7c15d48d922bb1.	Pins the o-linked project artifact used by the graph illustration.
Project DOT	13,065 bytes; SHA-256 bf16748f53a377e950a67b9c27d48ff260dcc987a69d86c5f29ca6289df7f00b; direct directory and lifted input byte-identical.	Confirms reversible project lifting at the compiler's logical graph boundary.
Rendered graph	53 semantic vertices, 62 directed edges; vector PDF SHA-256 75c7d219d208a2afddc51a5c9c682c506d26cd9729145828019f88f1c6a8e90e.	Pins the complete landscape figure and its reading guide.
python3 scripts/release_evidence.py validate	release evidence: PASS (26 required portable QEMU gates).	Validates the current evidence registry and its 26-gate portable projection.
Portable QEMU aggregate	26 of 26 required gates passed; 201 expected markers matched; 361.11 seconds.	Executes the complete current portable native evidence ladder, including every required Mode 22 through Mode 34 gate.

[Cargo.toml, lines 1–18] [Cargo.toml, lines 78–86] [CHANGELOG.md, lines 8–50] [mcp/ostadix_lang_mcp_server/src/main.rs, lines 1663–2207] [evidence/gates.toml, lines 1–4] [scripts/release_evidence.py, lines 487–586]

14.3 Exact reproduction commands

```
git rev-parse HEAD
# 36787b16476bc0c8c4ddf665c7228b314d04e716

cargo build --release --workspace --bins --all-features --locked

python3 scripts/release_evidence.py validate
# release evidence: PASS (26 required portable QEMU gates)

rg -n '\s*#\[tool\(' mcp/ostadix_lang_mcp_server/src/main.rs
# ten tool definitions

mkdir -p output/current-master
./target/release/o-link tests/fixtures/project_hgraph --project \
-o output/current-master/o-linked-project.0
./target/release/olangc output/current-master/o-linked-project.0 \
--target dot --route main --shim-dir backends \
> output/current-master/o-linked-codebase-hgraph.dot
```

```
./target/release/olangc tests/fixtures/project_hgraph \
  --target dot --route main --shim-dir backends \
  > output/current-master/project-hgraph-direct.dot
cmp output/current-master/o-linked-codebase-hgraph.dot \
  output/current-master/project-hgraph-direct.dot
cp output/current-master/o-linked-codebase-hgraph.dot \
  docs/figures/o-linked-codebase-hgraph.dot
dot -Tsvg output/current-master/o-linked-codebase-hgraph.dot \
  -o docs/figures/o-linked-codebase-hgraph.svg
dot -Tpdf output/current-master/o-linked-codebase-hgraph.dot \
  -o docs/figures/o-linked-codebase-hgraph.pdf
shasum -a 256 \
  output/current-master/o-linked-project.0 \
  output/current-master/o-linked-codebase-hgraph.dot \
  docs/figures/o-linked-codebase-hgraph.dot \
  docs/figures/o-linked-codebase-hgraph.svg \
  docs/figures/o-linked-codebase-hgraph.pdf
```

The current gate registry names the portable aggregate as:

```
./boot-and-test.sh smoke
```

That aggregate consumes the registry projection checked above. Individual mode scripts remain directly runnable and name their expected markers in `evidence/gates.toml`. This gives a reproducer one route from the release coordinate, through registry qualification, into each concrete hosted or QEMU execution transcript.

14.4 Coverage accounting

The current public surface is accounted for through explicit inventories: fourteen root binaries; one independent engine package; ten MCP tools; four current execution coordinates in Graph V2, Evidence V6, Admission V6, and Why V2; eight Information Bridge projections; three Registry V1 event bodies; four World wire families; thirteen required modes from 22 through 34; and the twenty-six-gate portable evidence registry. The sections and appendices map each inventory to defining source and the command that exposes it.

15 Implemented capabilities

- A structural polyglot syntax with exact language-tagged nesting, persistent environments, bindings, calls, quotation, and scopes.
- A rich typed interchange algebra with explicit fidelity metadata and a canonical CBOR wire protocol capped at 128 MiB per frame.
- Uniform hosted lowering through OIR, a validated plan, and a state-complete HGRAPH.
- Deterministic graph settlement with explicit resource/effect modeling and verified-inline parallelism.
- Deferred requests, Nix/OS values, and four first-class group topologies.
- Explicit route-preserving project lifting, reversible project bundles, logical Project HGraphs, executable route policies, and capacity-first Project Mesh placement.

- Hosted packaging to native host executables and WASI modules, plus separate Python and C17 reference/subset editions.
- A distinct freestanding O-CORE compiler and native x86-64/AArch64 kernel substrates, including challenged x86-64 BootInfo and bounded four-CPU startup.
- Hosted Live-World supervision and a strict KernelWorld record/admission contract.
- An independently packageable `ostadix-api` engine, an exact root compatibility shell, a unified lowercase `o` command, and a ten-tool local MCP.
- Graph V2 identity, Evidence V6 analysis, Admission V6 authority, Why V2 explanation, and one-use agent intent handles.
- Reciprocal SPAKE2 pairing, destination-issued credentials, pinned TLS, Registry V1 profiles, and generation-fenced mesh execution.
- Content-addressed Information, eight bounded native projections, typed provenance recovery, and canonical World identity, protocol, value, and receipt formats.

16 Design rationale

16.1 Why evaluator identity is syntax

I made evaluator identity part of syntax because the composition point is a semantic boundary. Structural recognition preserves native backend source while giving the host a complete heterogeneous program tree. The notation therefore carries both the foreign program and the information required to compose it.

16.2 Why graph state is explicit

Execution semantics include values, failures, ordering, actor mutation, and world effects. I represent those dimensions with ordinary values, completion tokens, and versioned resource nodes. This ontology preserves observable serial behavior while admitting a precise parallel frontier, making state-complete graph projection the architectural center of the hosted runtime.

16.3 Why analysis and authority are separate objects

I separate Graph V2 identity, Evidence V6 analysis, and Admission V6 authority because each stage answers a different technical question. Graph V2 states which solved computation exists. Evidence V6 states why each operation is executable in one exact context. Admission V6 proves that the context and analysis still reproduce at the execution boundary. Why V2 exposes the resulting dependency neighborhood in a stable reader-facing form. This separation gives compilers, local commands, agents, and placement systems one shared execution constitution.

16.4 Why the engine owns the types

I made `ostadix-api` the implementation owner so embedding is a primary runtime surface. Parser nodes, OVALUE, plans, graphs, evidence, admission, projects, and World records can be used directly without importing a command package. The root crate reexports those exact identities, preserving existing programs while keeping one evaluator and one value universe.

16.5 Why admission is typed across layers

I use one typed admission architecture across hosted and native execution. Hosted Live-World executes the lifecycle semantics and supplies a differential oracle for native World stages. Native M5 realizes an isolated service lifecycle on its own substrate. The connected authority path begins when KernelWorld binds verified package bytes, manifest claims, and policy into a deterministic record. The native parser verifies that record, Mode 20 constructs generation-bound admission objects from it, and Mode 21 consumes an admitted object at guest entry. This path carries authority through explicit verification products while the hosted and native lifecycle implementations share the same semantic vocabulary.

16.6 Why placement begins with capacity

Project Mesh begins with usable execution capacity because placement exists to increase the amount of work the language can execute. It ranks paired node profiles by capacity, then applies compatibility, generation, route, and policy constraints to the source-closed branch. Admission records, reciprocal TLS identity, and causal attempt traces make that capacity dependable at the moment it is used.

17 Conclusion

I designed OSTADIX-LANG around one proposition: evaluator identity belongs in the structural syntax of a typed expression. That decision gives the parser a heterogeneous program tree, gives OVALUE a typed boundary between independent runtimes, and gives OIR/HGRAPH the information required to represent dataflow, completion, actor state, and world effects explicitly. The nesting is the interface because the nesting survives every stage from source recognition to runtime settlement.

The project layer extends the same idea from expressions to complete source trees. Deterministic bundles preserve artifacts, route policies select whole execution alternatives, and the toolchain exposes their complete logical Project HGraph. Project Mesh closes a selected branch over its prerequisites, ranks paired nodes by capacity, and binds every attempt to node generations and returned artifacts.

O-CORE carries the architecture to the machine boundary. Typed source lowers through HIR and SSA MIR into freestanding x86-64 objects, which compose into a kernel with generation-tagged capabilities, isolated native services, strict KernelWorld records, and staged VM admission and entry. Hosted Live-World supplies executable lifecycle semantics for immutable packages, service generations, revocation, rollback, and reconstruction.

The result is one language system spanning recursive polyglot programming, state-complete graph execution, whole-project composition, native compilation, capability-bearing kernel services, Information, Registry, and World construction. The independent engine, unified o, and ten-tool MCP make the same runtime available to embedders, operators, and agents. Graph V2, Evidence V6, Admission V6, and Why V2 carry one continuous representation of evaluator choice, typed value, effect, explanation, authority, and execution substrate across all of those scales.

A Compact hosted-language reference

Form	Meaning
$\text{LANG}^{\sim}(\text{body})_ \text{LANG}$	Fresh ephemeral evaluator expression.

Form	Meaning
<code>LANG[n]^(body)_LANG[n]</code>	Persistent evaluator actor keyed by language, numeric ID, and process policy.
<code>LANG{lazy}^(...)_LANG{lazy}</code>	Cache-safe deferred inline evaluation; auto-forces on splice.
<code>LANG{defer}^(...)_LANG{defer}</code>	Non-cacheable deferred evaluation; requires explicit <code>now</code> .
<code>let x = expr</code>	Root binding store; right-hand side is a typed block or orchestration call.
<code>\$x</code>	Load and splice the visible binding.
<code>quote^(...)_quote</code>	Produce an unevaluated <code>OExpr</code> .
<code>O.eval(expr[, scope])</code>	Shim callback that reparses and evaluates quoted <code>O</code> source.
<code>scope(), O.scope()</code>	Capture the visible <code>O</code> lexical environment as an <code>OScope</code> .
<code>lazy(x), autonomous(x), now(x)</code>	Demand-policy and forcing operations.
<code>batch/all/any/race(...)</code>	First-class group construction with distinct settlement rules.
<code>instantiate, realise, activate</code>	Move through Nix expression, derivation, store path, and system rungs.
<code>dry_activate, current_system</code>	Construct an activation result without mutation; inspect the current system value.
<code>cap=, authority flags, effects=, reads=, writes=, serial=host</code>	Attach capability, effect, resource, and serialization metadata to a block.

A.1 Exact builtin signatures

Signature	Value transition
<code>instantiate(expr)</code>	<code>ONixExpr</code> to <code>ODerivation</code> .
<code>realise(drv)</code>	<code>ODerivation</code> to <code>OStorePath</code> .
<code>now(request_or_group)</code>	Force one request chain or settle one coordination group.
<code>dry_activate(path[, profile])</code>	Construct the dry activation result for one store path and optional profile.
<code>activate(path[, profile])</code>	Activate with ambient system authority.
<code>activate(system_activation_capability, path[, profile])</code>	Validate the typed activation capability and profile guard, then activate.
<code>current_system()</code>	Return the current referential <code>OSystem</code> .
<code>scope()</code>	Capture visible <code>O</code> bindings as a detached <code>OScope</code> .
<code>lazy(expr)</code>	Evaluate one expression under Lazy policy.
<code>autonomous(expr)</code>	Evaluate one expression under Autonomous policy and flush admitted work.
<code>batch(a,...), all(a,...), any(a,...), race(a,...)</code>	Capture an ordered member list with the selected settlement topology.

B Public command reference

Command	Principal forms	Result
<code>0</code>	<code>0 [-executor serial graph] [-workers N] [-backend-grant SPEC] [-json] [-check] FILE [shim-dir]; 0 -eval SOURCE or 0 -e SOURCE; -require-source-sha256 HEX; -require-execution-intent-sha256 HEX; 0 -repl [shim-dir]; no arguments in an interactive terminal; -i aliases -repl; 0 -version; 0 version [-json]; -h/-help; REPL commands :scope, :r, :?, :q</code>	Evaluate one document or enter and control the persistent interactive session.
<code>o dispatcher</code>	<code>run, plan, explain, inspect, why, kernel, node, node-host, registry, info, live, and receipt; all other forms enter 0</code>	Route one lowercase command surface into the compiled intent front door, compiler inspection, kernel, node, Registry, Information, Live-World, O-Git, or ordinary evaluator.
<code>o-cli</code>	<code>o run INPUT [execution options]; o plan INPUT [-route ID] [-live]; o explain [last-run RUN] [-json]; o inspect [last-run RUN] [-trace] [-json]</code>	Resolve one hosted or project intent, plan or execute it, and retain a durable run record for explanation and inspection.
<code>olangc</code>	<code>-target binary wasm script ir dot; -o/-output; -shim-dir; -keep-build-dir; repeatable -backend-grant; -route; -routes-policy; -project-trace-out; -grounding; -explain-schedule; -format; -execution-intent-json; -why; -workers; -world-id; -world-epoch</code>	Package, execute, or inspect the hosted graph. Script target executes immediately inside the compiler process.
<code>ocorec</code>	<code>One or more input modules; -emit ast hir mir asm obj; -target x86_64-unknown-none[-elf] or aarch64-unknown-none[-elf]; -o/-output; -keep-asm; -V/-version</code>	Inspect or compile the complete O-core pipeline.

Command	Principal forms	Result
o-link	Bare single-directory literal run; explicit -literal link; explicit -project project lift; -o/-output; -stdout; repeatable -lang; -verbose-skips; -no-validate; -run; -shim-dir; -backend-grant; -shebang; -list-routes; -route; -routes-policy; repeatable -route-decl; -project-trace-out; -parallel[=verified]; -parallel-required; -explain-parallel; -mesh[=prefer required]; -mesh-retries; -mesh-local-fallback; -mesh-discovery-timeout-ms; -mesh-no-lan-discovery; -mesh-peer-root; -mesh-trace-out; -explain-mesh	Create reversible project or literal artifacts and invoke selected routes.
Generated project binary	No argument for its selected default; -list-routes; -route ID; -routes-policy POLICY	Restore the embedded project and execute its route coordinator.
o-unlink	o-unlink input.0 (-o -output-dir) dir; -dry-run	Restore either embedded representation with format-specific validation.
o-notebook	o-notebook [shim-dir]	Serve the embedded loopback notebook and its shared evaluator session.
ogit	demo semantic-receipt; diff-semantic A B	Generate a semantic receipt or compare group policy and intent.
o-live-host	pack, install, activate, invoke, compose, upgrade, rollback, restart, status, demo	Drive the complete hosted lifecycle state machine.
o-node	start, stop, status, restart, pair, pki, identity, admin, profile, doctor, serve	Run the node service, establish reciprocal identity pins, and administer its bounded hosted execution state.
octl	node list, node use, node profile, node doctor, node run, node session, node authority	Discover and select paired nodes, invoke exact source, operate durable sessions, and issue development-authority leases.
o-registry	init, profile-local, publish-profile, verify, list, export, import	Operate the local signed Registry V1 event chain.
o-info	init, keygen, record, verify, import, head	Create, sign, verify, import, and inspect bounded content-addressed Information state.

Command	Principal forms	Result
ocore-kernel-world-record	-manifest; -payload; -o/-output; -V/-version	Verify one package/payload pair, encode its deterministic native admission record, and print its bytes, path, record hash, and package digest.
Python 0.4.0	python3 -m o_lang FILE; -as auto text html markdown latex json; -o/-output; -dump-ast; -V/-version; typed root API above	Execute the source-tree reference edition or consume typed Python API values.
C17 edition	c_cpp/0 INPUT.0 [shim-dir]; c_cpp/olangc INPUT.0 [-o OUT] [-shim-dir DIR] [-keep-build-dir]	Run or AOT-compile the standalone C17 hosted edition.
Setup and aggregate gates	setup.sh with -m/-minimal, -f/-full, -y/-yes, -v/-verify, -no-wrappers, -dry-run, -h/-help; boot-and-test.sh phases setup, hosted, ocore, kernel, smoke, tests, quick, full	Build, install, verify, or execute the repository-wide layer gates.
fuzz/parser	cargo fuzz run parser	Feed arbitrary bytes through the hosted parser under libFuzzer.

[src/main.rs, lines 15–80] [src/main.rs, lines 303–337] [scripts/o-cli.sh, lines 18–116] [src/bin/olangc.rs, lines 192–257] [crates/ostadix-api/src/ocore/driver.rs, lines 11–35] [src/bin/olink.rs, lines 155–374] [o_lang/cli.py, lines 123–141]

Thirteen root binaries expose **-h/-help** at their public command boundary. The notebook serves its positional loopback interface at 127.0.0.1:8888. The historical C17 help behavior is recorded in the final boundary table.

Every stateful **o-live-host** command accepts global optional **-policy FILE**. Its exact subcommand arguments are:

```
pack --manifest MANIFEST --payload PAYLOAD
install --state STATE --manifest MANIFEST --payload PAYLOAD [--alias ALIAS]
activate --state STATE DIGEST
upgrade --state STATE DIGEST
invoke --state STATE SERVICE PROTOCOL OPERATION INPUT
compose --state STATE --plan PLAN --input INPUT
rollback --state STATE PACKAGE
restart --state STATE SERVICE
status --state STATE
demo --state STATE
```

C Example atlas

The example atlas has ten families. The 42 tracked top-level **examples/*.0** files form nine executable teaching families. Companion fixtures in **examples/group_pipeline/**, **tests/fixtures/project_hgraph/**, and **tests/fixtures/project_hgraph_exec/** form the tenth through semantic receipts, logical project graphing, and bounded project execution.

Family	Representative files	Language fact
Core structure	hello, bindings, nested_splice, trailing_expr	Typed blocks, O bindings, structural nesting, and backend result extraction.
HTML and typed rendering	html_basic, html_python_html, html_escape, html_raw_roundtrip, python_html_python	Receiving evaluator projection, trusted HTML, escaping, and three-language nesting.
State and documents	computed_plot, literate_report, persist, env_split, ephemeral, script	Images, literate output, actor identities, fresh environments, and shebangs.
Meta-evaluation	meta_eval	Quotation, O expressions, callbacks, and explicit lexical scope.
Shell and JavaScript	bash_*, shell_hello, js_*	Host-language execution, binding injection, JSON, and multiline programs.
SQL	sql_create_insert_ select, sql_select, sql_aggregation, sql_python_sql	Persistent SQLite actors and SQL/Python value circulation.
Nix lattice	nix_basic, nix_python_html, nix_storepath*, instantiate_realise_ basic	Nix expressions, derivations, store paths, and cross-language projection.
Requests and groups	lazy_defer_attrs_basic, lazy_request_basic, coordination_groups	Demand policy, forcing, cache safety, and four settlement topologies.
Operating systems	os_as_participant_basic, nixos_test*	OSystem values, activation boundary, and NixOS test records.
Composition fixtures	examples/group_ pipeline/ tests/fixtures/project_ hgraph/, tests/fixtures/ project_hgraph_exec/	Semantic receipts, the exact published Project HGraph, prerequisite and alternative branches, and bounded project execution settlements.

D Source map

Concern	Primary source	Why it is authoritative
Hosted grammar	crates/ostadix-api/ src/parser.rs	AST, exact tag parser, contexts, aliases, escapes, and parser tests.
Hosted execution	crates/ostadix-api/ src/eval.rs	Policies, requests, groups, lowering entry, dispatch, scopes, authority, and rendering.

Concern	Primary source	Why it is authoritative
Cross-language values	<code>crates/ostadix-api/src/value.rs</code> , <code>crates/ostadix-api/src/wire.rs</code>	Tagged algebra, fidelity, identities, requests, runtime boundaries, and CBOR.
OIR and plan	<code>crates/ostadix-api/src/ir.rs</code>	Lowered forms, dependencies, backend interfaces, registry, and renderers.
HGraph	<code>crates/ostadix-api/src/hgraph/</code>	Hypergraph ontology, plan projection, solving, validation, and schedule.
Effects and coordination	<code>crates/ostadix-api/src/effects.rs</code> , <code>crates/ostadix-api/src/executor/</code>	Resources, conservatism, worker eligibility, atomic settlement.
Backend process model	<code>crates/ostadix-api/src/process.rs</code> , <code>crates/ostadix-api/src/backend.rs</code>	Self-exec actors, sandbox policy, adapter invocation, and decoding.
Projects	<code>crates/ostadix-api/src/project/</code> , <code>src/bin/olink.rs</code>	Bundle model, materialization, routes, policies, lifting, and restoration.
Hosted compiler/tooling	<code>src/bin/olangc.rs</code> , <code>Cargo.toml</code>	Declared binaries and exact behavior of every target.
O-core	<code>crates/ostadix-api/src/ocore/</code> , <code>docs/OCORE.md</code>	Lexer through object generation and explicit compiler boundaries.
Native kernel	<code>ocore/kernel/</code> , <code>ocore/runtime/x86_64/</code>	Boot/link/runtime code and substrate-specific execution gates.
Hosted Live-World	<code>crates/ostadix-api/src/live_system/</code> , <code>docs/HOSTED_LIVE_REFERENCE.md</code>	Executable supervisor and explicit hosted-oracle boundary.
KernelWorld	<code>crates/ostadix-api/src/kernel_world.rs</code> , <code>ocore/runtime/x86_64/kernel_world_admission.oc</code>	Strict host record, independent native parser/policy, and generation-bound admission.
Runtime ownership and intent	<code>crates/ostadix-api/src/lib.rs</code> , <code>src/lib.rs</code> , <code>src/bin/o-cli.rs</code>	Independent engine ownership, exact compatibility reexports, unified intent, and durable run records.
Execution constitution	<code>crates/ostadix-api/src/evidence/</code>	Graph V2, Evidence V6, Admission V6, Why V2, and current coordinator authority.
Nodes and Project Mesh	<code>crates/ostadix-api/src/hosted_remote/</code>	Pairing, pinned TLS, node profiles, capacity ranking, placement, retry, and attempt traces.

Concern	Primary source	Why it is authoritative
Information and provenance	<code>crates/ostadix-api/src/information/</code> , <code>crates/ostadix-api/src/information_bridge/</code> , <code>crates/ostadix-api/src/information_provenance/</code>	Content-addressed state, eight native projections, and typed recovery admission.
Registry and World	<code>crates/ostadix-api/src/registry/</code> , <code>crates/ostadix-api/src/world/</code>	Signed namespace/profile chains and the canonical World wire families.
Agent and native evidence	<code>mcp/ostadix_lang_mcp_server/</code> , <code>evidence/gates.toml</code>	Ten local agent tools and the required portable native gate registry.

E Complete ostadix-api module inventory

The independent engine declares 42 modules. Thirty-six are public and are reexported by the root compatibility library. Six are crate-internal implementation modules: `backend_catalog`, `canonical_cbor`, `capability`, `dispatch_model`, `eval_core`, and `placement_protocol`. The table accounts for every declaration in the engine root exactly once.

Concern	Declared modules
Language and data model	<code>api</code> , <code>parser</code> , <code>syntax_dialect</code> , <code>value</code> , <code>version</code> , <code>wire</code> , <code>canonical_cbor</code> (internal).
Hosted execution and backends	<code>backend</code> , <code>backend_catalog</code> (internal), <code>backend_morphism</code> , <code>backend_state</code> , <code>capability</code> (internal), <code>dispatch_model</code> (internal), <code>effects</code> , <code>environment</code> , <code>eval</code> , <code>eval_core</code> (internal), <code>execution_contract</code> , <code>executor</code> , <code>hgraph</code> , <code>ir</code> , <code>process</code> , <code>runtime_exec</code> , <code>scheduler</code> , <code>shims</code> .
Evidence, intent, placement, and projects	<code>evidence</code> , <code>intent</code> , <code>placement</code> , <code>placement_protocol</code> (internal), <code>project</code> .
Distributed state and information	<code>hosted_remote</code> , <code>information</code> , <code>information_bridge</code> , <code>information_provenance</code> , <code>registry</code> , <code>resource_identity</code> , <code>world</code> .
Lifecycle and native systems	<code>kernel_world</code> , <code>live_system</code> , <code>nix_ops</code> , <code>nixos_ops</code> , <code>ocore</code> .

[`crates/ostadix-api/src/lib.rs`, lines 9–59] [`src/lib.rs`, lines 7–13]

F Architectural position among neighboring systems

The complete contribution joins expression-granular evaluator selection, recursive evaluator nesting, independently implemented runtimes, typed OVALUE exchange, OIR/plan/HGraph lowering, V6 evidence-based admission, capacity-first project placement, content-addressed Information, canonical World records, and an O-core path from typed native source into capability-bearing World execution.

Neighboring family	Its organizing unit	Ostadix-lang organizing unit
Racket #lang and language-oriented programming	One declared module language in a common host substrate.	A recursively nested evaluator-indexed expression dispatched to an independently implemented runtime.
Polyglot notebooks and literate tools	A top-level cell or document block.	An AST expression at arbitrary nesting depth, including Python inside HTML inside Python.
Staged metaprogramming	Code represented as data inside a staged language system.	<code>OExpr</code> , <code>quote^</code> , lexical <code>OScope</code> , and <code>O.eval</code> re-enter heterogeneous evaluator structure.
String-embedded DSLs	A string whose integration is defined by its host language.	A parsed typed-expression boundary whose result is an OVALUE consumed by the receiving evaluator.
Workflow engines	Tasks and edges in an external workflow model.	Requests, groups, policies, values, and effects inside the language's value and graph model.
Eco, PyHyp, and GraalVM	Nested editor boxes, an engineered language pair, or a shared polyglot runtime substrate.	Registry-extensible expression tags, independent runtimes, typed exchange, and whole-program state-complete graph lowering.
Freestanding systems languages	Native compilation and kernel construction.	O-core's typed HIR/SSA MIR pipeline inside a hosted-to-native World authority chain.

Primary reference points. The neighboring organizing models are documented in Felleisen et al., *A Programmable Programming Language*; the [Jupyter messaging specification](#); the [Org Babel language manual](#); Diekmann and Tratt, *Eco: A Language Composition Editor*; Barrett et al., *Fine-grained Language Composition: A Case Study*; and the [GraalVM Polyglot embedding reference](#). These sources define the comparison units used in the table: module language, notebook or document block, language box, engineered language pair, and shared polyglot runtime.

The implementation makes every junction explicit. The parser preserves evaluator identity; OVALUE preserves cross-language meaning; the plan freezes backend interfaces; HGRAPH exposes value and resource state; project policies settle whole-tree alternatives; and exact records carry verified host intent into native admission.

G Reproduction commands

The commands below begin at repository root and preserve the same source coordinate used throughout the paper.

G.1 Pin the source and qualify the Rust workspace

```
git rev-parse HEAD
# 36787b16476bc0c8c4ddf665c7228b314d04e716

cargo fmt --all --check
cargo check --workspace --all-targets --all-features --locked
cargo clippy --workspace --all-targets --all-features --locked -- -D warnings
OSTADIX_TEST_RUNTIME_POLICY=required \
  cargo test --workspace --all-targets --all-features --locked
cargo build --release --workspace --bins --all-features --locked
```

G.2 Run the language and unified front door

```
./target/release/0 examples/hello.0 backends
./scripts/o-cli.sh run examples/hello.0
./scripts/o-cli.sh plan examples/hello.0
./scripts/o-cli.sh explain last-run
./scripts/o-cli.sh inspect last-run

for bin in 0 o-cli olangc ocorec o-link o-unlink ogit \
  o-live-host o-node octl o-registry o-info \
  ocore-kernel-world-record
do
  "./target/release/$bin" --help
done
```

The notebook owns a positional launch surface and can be exercised through its loopback API:

```
./target/release/o-notebook backends
curl -fsS http://127.0.0.1:8888/
curl -fsS -X POST http://127.0.0.1:8888/eval \
  -H 'content-type: application/json' \
  --data '{"code":"text^(42)_text"}'
curl -fsS -X POST http://127.0.0.1:8888/reset
```

G.3 Qualify the independent engine and MCP

```
cargo test -p ostadix-api --all-targets --all-features --locked

cargo build --release --locked \
  --manifest-path mcp/ostadix_lang_mcp_server/Cargo.toml
cargo clippy --all-targets --locked \
  --manifest-path mcp/ostadix_lang_mcp_server/Cargo.toml -- -D warnings
cargo test --locked \
  --manifest-path mcp/ostadix_lang_mcp_server/Cargo.toml
python3 scripts/smoke_ostadix_mcp.py
rg -n '\s*#[tool\(' mcp/ostadix_lang_mcp_server/src/main.rs
```

G.4 Generate and read the o-linked Project HGraph

```
mkdir -p output/current-master
./target/release/o-link tests/fixtures/project_hgraph --project \
  -o output/current-master/o-linked-project.0

./target/release/olangc output/current-master/o-linked-project.0 \
  --target ir --route main --shim-dir backends \
  > output/current-master/o-linked-project.ir
./target/release/olangc output/current-master/o-linked-project.0 \
  --target dot --route main --shim-dir backends \
  > output/current-master/o-linked-codebase-hgraph.dot
./target/release/olangc tests/fixtures/project_hgraph \
  --target dot --route main --shim-dir backends \
  > output/current-master/project-hgraph-direct.dot
cmp output/current-master/o-linked-codebase-hgraph.dot \
  output/current-master/project-hgraph-direct.dot
cp output/current-master/o-linked-codebase-hgraph.dot \
  docs/figures/o-linked-codebase-hgraph.dot
dot -Tsvg output/current-master/o-linked-codebase-hgraph.dot \
  -o docs/figures/o-linked-codebase-hgraph.svg
dot -Tpdf output/current-master/o-linked-codebase-hgraph.dot \
  -o docs/figures/o-linked-codebase-hgraph.pdf
```

G.5 Run Project HGraph, Live-World, and World gates

```
./scripts/smoke-project-hgraph.sh
./scripts/smoke-project-hgraph-exec.sh
./scripts/smoke-hosted-live-reference.sh

python3 -m unittest -v \
  tests.test_release_evidence \
  tests.test_source_release \
  tests.test_example_manifest

python3 scripts/release_evidence.py validate
./boot-and-test.sh smoke
```

The aggregate reads the current 26-gate portable projection from `evidence/gates.toml`. Each registered script emits the markers attached to its gate entry.

G.6 Run compiler targets

```
./target/release/olangc examples/hello.0 --target script \
  --shim-dir backends
./target/release/olangc examples/hello.0 --target ir \
  --shim-dir backends > output/current-master/hello.ir
./target/release/olangc examples/hello.0 --target dot \
  --shim-dir backends > output/current-master/hello.dot
./target/release/olangc examples/hello.0 --target binary \
  --shim-dir backends -o output/current-master/hello-host
./output/current-master/hello-host
```

```
./target/release/olangc examples/hello.0 --target wasm \
  --shim-dir backends -o output/current-master/hello.wasm
wasmtime output/current-master/hello.wasm
# expected current-host result: nonzero
# Error: failed to evaluate program
# Caused by: failed to locate current 0 executable
```

G.7 Run the Python and C17 editions

```
python3 -m tests.test_parser
python3 -m tests.test_evaluator
python3 -m compileall -q o_lang

make -C c_cpp
make -C c_cpp test
make -C c_cpp olangc-test
make -C c_cpp warnings-as-errors
cmake -S c_cpp -B target/c17-cmake
cmake --build target/c17-cmake
ctest --test-dir target/c17-cmake --output-on-failure
```

G.8 Inspect Information, Registry, and node surfaces

```
./target/release/o-info --help
./target/release/o-registry --help
./target/release/o-node --help
./target/release/octl --help
```

The type-level and state-transition suites for Information, Bridge, Provenance, Registry, pairing, TLS, Project Mesh, and World run inside the all-target, all-feature Rust matrix above.

H Current host, integration, and proof boundaries

These are the current-master boundaries a reader could reasonably confuse with the mechanisms established in the body of the paper.

Boundary	Exact current state
Lowercase front door	The compiled <code>o-cli</code> owns <code>run</code> , <code>plan</code> , <code>explain</code> , and <code>inspect</code> . The repository Bash dispatcher owns routing for the other command families. On a case-insensitive filesystem, <code>PATH</code> order must select the installed lowercase wrapper ahead of the uppercase <code>O</code> binary.
<code>o-link</code> directory behavior	A bare single directory literal-links and runs. Explicit <code>-literal</code> produces a link-only hosted artifact unless paired with <code>-run</code> . Explicit <code>-project</code> produces the route-preserving inert project bundle used by the graph in this paper.

Boundary	Exact current state
Run records	<code>RunRecordV1</code> is a content-bound local observation record. It carries no signature or remote attestation and therefore derives trust from the local store and the process that produced it.
MCP execution	The analyze handle is process-local, expiring, one-use, and bound to its program, path, root, backends, source, and intent. The server consumes it before root-runtime validation. The direct <code>o_run</code> tool is a separate direct execution route and carries no analyze-handle gate.
MCP Python assertion	The isolated MCP Rust package passed 29 tests, and the real ten-tool smoke passed. The separate root Python <code>tests.test_mcp_smoke</code> suite passed four assertions and failed one brittle build-command string assertion because its expected command omits the current <code>-bin o-cli</code> argument.
WASI execution	<code>olangc -target wasm</code> successfully generated the <code>wasm32-wasip1</code> module. The current Wasmtime launch on this host then attempted to locate <code>0</code> and failed at that runtime lookup. The paper claims current WASI generation from this run.
Pairing and TLS	The outbound peer record pins the destination CA and server name. The inbound listener validates a client chain against the pairing CA. The current inbound policy has no per-leaf allowlist or revocation service beyond that CA validation and stored pairing state.
Discovery	<code>o node list</code> reports UDP discovery advertisements. Paired TLS connection and an application request establish the stronger operational facts used by Project Mesh.
Project Mesh	Mesh transfers a source-closed route branch and its project prerequisites. Migration replays declared actor generations and project state contracts. It does not move live process memory.
Information provenance	A provenance sidecar establishes origin only when admission recomputes its exact image and the recovery assessment reports Exact . Outstanding producer, procedure, authorization, receipt, attribution, execution, effect, or morphism obligations retain an unestablished origin.
Information Bridge	The eight projections are bounded metadata records. They carry no credential, capability bearer, mutation handle, dispatch handle, or generic native-value serialization.
Registry V1	The registry implementation verifies append-only local event chains, delegations, signatures, freshness, rollback, and equivocation. It defines no network publication or discovery service by itself.
World formats	<code>OWIDENT</code> , <code>OWPROTO</code> , <code>OWVALUE</code> , and <code>OWRECEIPT</code> are canonical record formats. Capability identities and rights inside them are descriptive. Signer authorization, key enrollment, revocation, and current Governor state come from the policy layer that consumes a verified record.
Evidence registry	<code>release_evidence.py validate</code> validates the registry and its required projection; it is not a QEMU transcript. The separate current aggregate execution completed 26 of 26 required portable gates and matched 201 expected markers.

Boundary	Exact current state
Mode 21	Mode 21 is the supplemental hardware-KVM gate and lies outside the required Modes 22 through 34 set. Its AMD SVM/NPT and writable <code>/dev/kvm</code> requirements make it host-dependent. It was not executed in this host audit because that required hardware substrate was unavailable.
Native scope	The registered native gates establish their exact bounded QEMU and hosted contracts. Physical-machine execution, Secure Boot, measured boot, general foreign-kernel boot, general Linux or Plan 9 compatibility, PCI assignment, DMA isolation, and IOMMU isolation require their own evidence gates.
Fuzz package lock	<code>cargo check --manifest-path fuzz/Cargo.toml</code> passed, then Cargo rewrote the stale tracked <code>fuzz/Cargo.lock</code> . The audit restored that disposable worktree exactly. This is a locked-fuzz reproducibility defect in the tracked lock projection.
Release all-target selection	<code>cargo build --release --workspace --bins --all-features --locked</code> passed. Adding <code>--all-targets</code> is not a passing release-build command at this snapshot: integration-harness compilation reaches <code>tests/hosted_remote_v2.rs</code> , which calls two runtime injection methods gated by <code>cfg(debug_assertions)</code> and therefore absent from the release dependency. Release artifacts use the complete binary target set; test targets use the separately passing all-target, all-feature test command.
World-alpha G0	The World-alpha G0 lane reached its historical continuity prerequisite and stopped there. The current portable 26-gate registry and Modes 22 through 34 results do not promote that history-dependent lane into a G0 passage.
Multipass Linux lane	The requested Multipass Linux lane was unavailable in the current host audit. Linux-specific results in this paper therefore come from the registered portable QEMU gates and the bounded container probe.
Live-World state	Hosted Live-World status reconstructs from the revisioned active set, immutable CAS, and policy. Rollback swaps the one retained healthy root. The public command set has no <code>rebuild</code> operation and maintains no lifecycle journal.

[scripts/o-cli.sh, lines 18–26] [crates/ostadix-api/src/intent/record.rs, lines 53–100] [mcp/ostadix_lang_mcp_server/src/main.rs, lines 43–187] [mcp/ostadix_lang_mcp_server/src/main.rs, lines 1858–2015] [crates/ostadix-api/src/hosted_remote/v2/runtime.rs, lines 740–773] [tests/hosted_remote_v2.rs, lines 2300–2432] [crates/ostadix-api/src/hosted_remote/tls.rs, lines 204–303] [crates/ostadix-api/src/information_bridge/mod.rs, lines 1–54] [crates/ostadix-api/src/information_provenance/mod.rs, lines 330–505] [crates/ostadix-api/src/registry/verify.rs, lines 227–348] [crates/ostadix-api/src/world/receipt.rs, lines 600–769] [evidence/gates.toml, lines 1–4] [scripts/release_evidence.py, lines 487–586]